

Heron: A Knowledge Graph editor for intuitive implementation of python based experimental pipelines

Reviewed Preprint

v2 • February 24, 2025

Revised by authors

Reviewed Preprint

v1 • March 5, 2024

George Dimitriadis , Ella Svahn, Andrew MacAskill, Athena Akrami

Sainsbury Wellcome Centre, University College London, London, United Kingdom • Department of Neuroscience, Physiology and Pharmacology, University College London, London, United Kingdom

 https://en.wikipedia.org/wiki/Open_access
 Copyright information

eLife Assessment

This **valuable** paper introduces Heron, lightweight scientific software that is designed to streamline the implementation of complex experimental pipelines. The software is tailored for workflows that require coordinating many logical steps across interconnected hardware components with heterogeneous computing environments. The authors **convincingly** demonstrate Heron's utility and effectiveness in the context of behavioral experiments, addressing a growing need among experimentalists for flexible and scalable solutions that accommodate diverse and evolving hardware requirements.

<https://doi.org/10.7554/eLife.91915.2.sa2>

Abstract

To realise a research project idea, an experimenter faces a series of conflicting design and implementation considerations, regarding both its hardware and software components. For instance, the ease of implementation, in time and expertise, should be balanced against the ease of future reconfigurability and number of 'black box' components. Other, often conflicting, considerations include the level of documentation and ease of reproducibility, resource availability as well as access to online communities. To alleviate this balancing act between opposing requirements we present Heron, a new Python-based platform to construct and run experimental and data analysis pipelines. Heron's main principle is to allow researchers to design and implement the experimental flow as close as possible to their mental schemata of the experiment, in the form of a Knowledge Graph. Heron is designed to increase the implementation speed of experiments (and their subsequent updates), while minimising the number of incorporated black box components. It enhances the readability and reproducibility of the final implementation and allows the use of combinations of hardware and software otherwise impossible or too costly to achieve. Given this, Heron offers itself to sciences whose needs involve experiments with a large number of interconnected hardware and software components like robotics, neuroscience, behavioural sciences, physics, chemistry, environmental science, etc.. It is designed with those experimentalists in mind which: i) Demand full control of their setup. ii) Prefer not to have to choose between hardware and software that run only on a specific chip/operating system

combination. iii) Appreciate the ease and speed that high-level languages (e.g. Python) and Graphical User Interfaces (GUIs) offer them. It assumes an intermediate knowledge of the Python language and ecosystem, offering a flexible and powerful way to construct experimental setups. It removes any inaccessible corners, yet keeps implementation costs significantly reduced compared to using lower level languages. Finally, its use results in a much cleaner and easier to understand code base, amicable to documentation and reproducibility efforts.

Introduction

One can divide an experiment's life circle, from concept to a running system, into a number of transformations. At first, the scientific question is mapped into an abstract schema of experimental steps (i.e. what needs to happen in order to answer the question). Subsequently these conceptual steps are transformed into a schema of hardware connectivity and software logic. At this stage the experimenter thinks in terms of high level objects like cameras and other types of sensors, time lines, triggers and events, agents and rewards, inputs and outputs. The final challenging step, which is rarely addressed (or even cognitively acknowledged) is to map the schema of the hardware and software logic to the actual hardware connectivity and operational code bases. At this level the experimenter has to work with much lower level objects like voltage differences, light intensities, TTL pulses, GPU shaders and information flow loops. This last and most time consuming step, can limit the number of iterations for ideas to be piloted and tested. Once the mental schema has been translated into code, it is this code that is usually addressed by other experimentalists in reproducibility efforts. These efforts usually face a high barrier in understanding the original experimental schema starting from its code implementation. This is because efficiently translating many lines of code (even if well documented) back to what the code actually accomplishes is in most cases a difficult task that requires years of coding experience, as well as familiarity with experimental designs. This barrier hinders efforts of reproducibility and quality control and is one that cannot be addressed solely by open sourcing one's work. Finally, the complexity of the translation from mental schema to running code-base makes design iteration efforts practically impossible. The prohibitively large iteration time, on one hand, and the interdependency of engineering decisions throughout the implementation cycle, on the other, makes updates of the experimental flow extremely cumbersome. This often results in practically one of the most serious experiment design and implementation hurdles. The need to create radically new implementations for only small changes in the underlying mental schema. These arguments can be seen as the main driver for the development and the rapid acceptance of high-level languages in software engineering and of micro-controller kits (e.g. Arduino) in electrical engineering and robotics. Concepts like object oriented programming (1 [↗](#)) or actor based programming (2 [↗](#)) for example, have been nothing else but an effort to take away the low level concepts that one needs to ultimately manipulate, and replace them with bundles of higher level ones, easier to cognise with.

In order to address this discrepancy between the mental schemata and implementation outcomes in experimental construction, we developed Heron, an open source software (MIT license) platform for the construction of data flow pipelines (e.g. experiments, data analysis, robotics, etc.). Heron comes with a series of distinguishing features. The primary one is that it creates experimental pipelines that visually and structurally, bear a very significant resemblance to the original mental schema of the experimental pipeline. So what one gets as the final experiment implementation is both semantically and syntactically very close to how one originally envisions the experiment should work. Because of that, Heron creates final implementations that are easy to understand, construct, communicate and change. In this way it often makes it fairly easy to put together helpful diagrams and documentation by following Heron's visual representation of the experiment and its underlying code. It also allows for accurate inference of the real time complexity of any proposed change even before any new code is written.

A second distinguishing feature of Heron is that it targets the experimenter without expertise in arcane subjects like networking, hardware connectivity or low level software - hardware interactions. By abstracting away these low level features in its Graphical User Interface (GUI) it allows construction of experiments with multiple, diverse hardware components, even using networks of computers running different operating systems, without bothering the experimenter with having to deal with all the low level connectivity issues that arise. It achieves this without limiting its users to preconceived ideas of how any single specific piece of hardware or code should be used. This is possible as Heron offers users an Application Programming Interface - API - letting them write the code that implements their own ideas at the optimal level of abstraction given the situation. Offering Python as the main (but not only) language for this user-centric code implementation, Heron makes it even easier for code novices to achieve highly complex experimental setups that are easy to both construct and reconfigure. In the following section, we will focus on Heron as a general purpose tool for constructing pipelines used to conduct different types of experiments. First, We will describe the specific meaning of an experimental pipeline implemented as a Knowledge Graph (7 [↗](#)). We then catalogue the design benefits and distinguishing features offered by Heron, in comparison to other efforts targeting the construction of experiments. Subsequently, we will describe the internal architecture of Heron with enough details to allow any developer to quickly get up to speed with Heron's code and contribute to its open source. Finally, we will illustrate a number of Heron experimental implementations, currently in use in the lab, each showcasing one of Heron's special features.

Concepts and Background

Mental schemata, Knowledge Graphs and Heron processes. The philosophy behind Heron's design

A mental schema is a psychological concept (5 [↗](#)), (6 [↗](#)) that is meant to define the way humans cognise. According to it, when people think, they categorise sensory experiences and abstract notions into groups. They then utilize the relations between these groups to draw conclusions about some hypotheses. Upon inputs from the environment and prior cognitive outcomes, the categories and their relations can update fluidly (6 [↗](#)). An example, for the case of interest here, is the mental schema of an experiment. In order for an experiment to be developed, the experimenter brings together a set of categories, both based on their sensory experiences (e.g. a laser, a data acquisition board, or a camera) and on abstract notions (e.g. time concurrency, or subject's choices). Then a set of relationships is generated between them. The sets of these concepts with their interactions defines the mental schema of the specific experiment. For example, a camera frame must be captured immediately after a specific event has taken place. Yet, today's implementations of experimental pipelines are written (with visual or text-based code) such that they obfuscate the mental schemata they derive from. A receiver of such a code base, irrespective of their understanding of the underlying language, always needs a significant amount of time and mental effort to map back the initial mental schema. A Knowledge Graph (KG) (7 [↗](#)) is a mathematical structure designed to capture the unstructured human knowledge on a subject (i.e. the mental schemata of different individuals relating to the same knowledge corpus) in such a way that a machine could use it to test propositions against the knowledge and also create novel propositions that a human with the knowledge would find to be true. It is practically an effort to implement the fuzzy notion of a mental schema into a concrete structure of objects and relations that is machine implementable. The fundamental structures of a KG are its nodes and their attributes. Nodes are meant to represent a group of objects at a desired level of abstraction, which does not have to be uniform amongst the different nodes of the same KG. Nodes' attributes define the state of each node and the edges that connect different nodes representing their relationships. Nodes and their attributes usually have semantic labels.

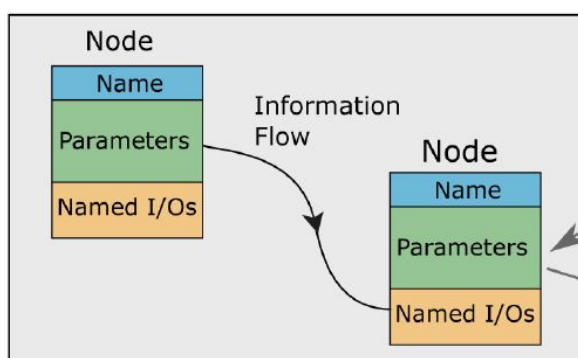
Heron implements an experiment in the form of a Knowledge Graph and does so at two separate levels. One is the graphical level, where a series of Nodes and their in-between links (Edges) are defined. The second is the code that defines each Node's functionality. The graphical level is used to construct a Knowledge Graph representing the experiment. The text-based code level is used to implement the dynamics of each Node in the Graph and define its level of abstraction. The Knowledge Graph's Nodes are labelled and have human readable attributes partially defining their state. They are also connected with directed Edges (links) through their named input and output points. The Edges represent message passing between Nodes (see [Figure 1](#)). In this way a graphical representation of an experimenter's mental schema is created with each Node at the appropriate level of abstraction for the specific experiment. This individualised level of abstraction is achieved through Heron's second level of implementation, i.e. a text based code that defines each Node's functionality. This is an important distinction between Heron and other node-based software like LabView and Bonsai (see Background section). Heron expects that each Node's behaviour and connectivity is defined by the experimenter, in normal, text-based code, and comes with an appropriate API to facilitate this. In this way Heron does not offer a long list of predefined Nodes which would make very strong assumptions about the structure of the experiment's mental schema. Instead, it offers the tools for designing and implementing one's own Nodes in a case by case approach at a level of abstraction required at any time. For instance, if in one case a Node representing a camera is required, while in another a Node should represent a large group of synchronised cameras acting as one, Heron provides the tooling to create either of these Nodes with minimum effort. Heron's code base is implemented in Python. This choice was informed by the need to keep the code simple to implement while at the same time be able to interact with a very diverse list of hardware drivers and software analysis libraries (see Design benefits section). It also means that Python is the go to language for the user's creation of new Node behaviours. Yet the language that Heron itself is implemented in (Python) does not enforce the language new Nodes can be implemented in by Heron users. Heron offers users a comfortable starting point to implement their own Nodes (in Python), while allowing for expert users to utilise lower level languages that offer other advantages like faster run times and lower level control of a machine's components (e.g. CPUs, GPUs, RAM, etc.). Heron poses very few (if any) limitations in what types of behaviour can be implemented in any Node. Breaking up the implementation into two levels of Knowledge Graph and running codes, as mentioned above, confers a much required "break complex problems into simpler ones" functionality at the heart of Heron's operation.

Background

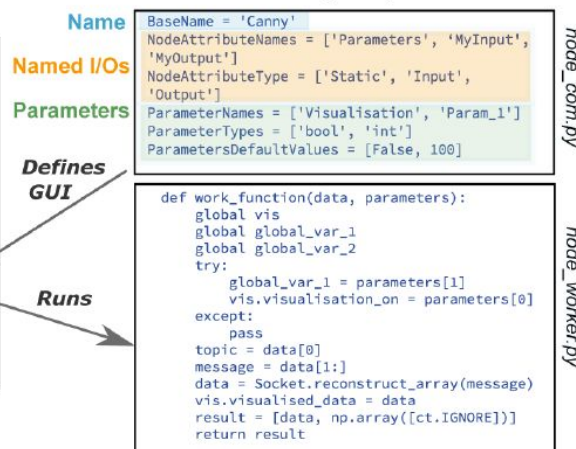
Computational graphs

Heron's Knowledge Graph approach has its root in a number of software frameworks where experimental and data analysis pipelines are constructed as a computational graph. MATLAB Simulink ([18](#)) and LabVIEW ([17](#)) are two of the more well-known frameworks where experiments can be designed as computational graphs using pre-defined elements (i.e. nodes provided by the developers of these systems). Bonsai ([4](#)) is a new entry to this field originally designed to ease the implementation of neuroscience and behavioural science experiments. The use of directed acyclic graphs (DAGs) ([19](#)) has increased dramatically in the Big Data analytics tools where frameworks like Apache Air-flow ([20](#)) and Dask ([21](#)) allow parallelisation of algorithms and data queries over large clusters of machines. Two efforts that are very similar to Heron in the way they structure computational graphs to define experimental pipelines are EPypes ([8](#)) and the Robot Operating System (ROS) ([9](#)). EPypes is (according to the developers own description) a "Python-based software framework for developing vision algorithms in a form of computational graphs and their integration with distributed systems based on publish-subscribe communication". The basic idea of message passing between individual processes, each responsible for its own algorithm, running on different machines is identical to Heron (even at the level of using the publish-subscribe communication protocol), although EPypes's focus is on

A Heron's Node Editor



Code Editor (IDE)



B Heron's Node Editor; Actual GUI

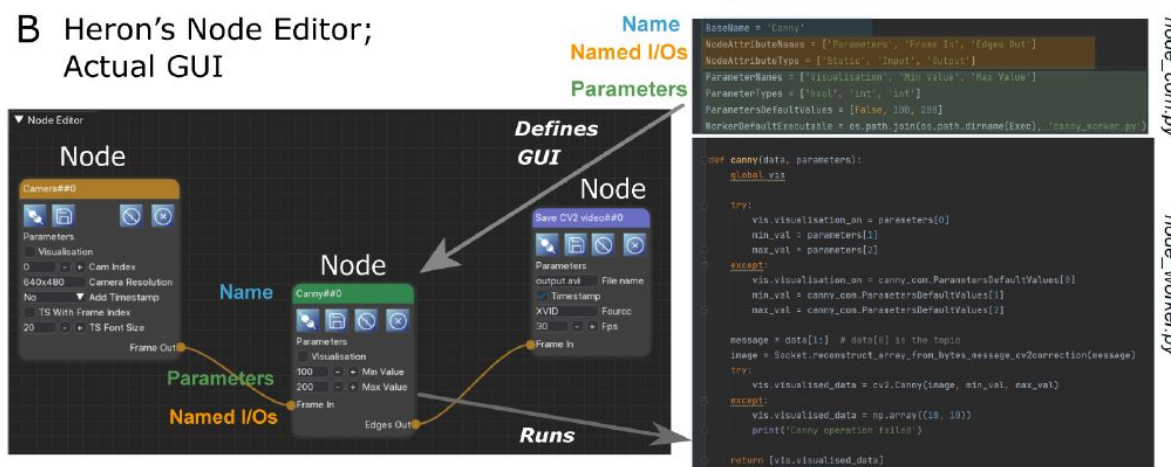


Fig. 1.

A. Schematic of Heron's two levels of operation. Left shows a Knowledge Graph made of Nodes with Names, named Parameters and named Input/Output points. Nodes are connected with links (Edges) that define the flow of information. Right shows a piece of textual code that fully defines both the Node's GUI appearance and the processes run by the machine. B. Heron's implementation of A, where the Knowledge Graph is generated in Heron's Node Editor window while the code that each Node needs to be defined and the code that each Node is running is written in a separate Integrated Development Environment (IDE, in this case PyCharm).

computer vision algorithms. The exact same idea is utilised by ROS where (in their own words again) “The ROS runtime “graph” is a peer-to-peer network of processes (potentially distributed across machines) that are loosely coupled using the ROS communication infrastructure. ROS implements several different styles of communication, including synchronous RPC-style communication over services, asynchronous streaming of data over topics, and storage of data on a Parameter Server.”

Hybrid programming

Heron’s approach is based on an existing programming idea, i.e. combining both visual and textual programming in a hybrid manner. One of the most widely used examples of such realisations is VVVV ([10](#)), a framework utilising visual programming with the C# or HLSL programming languages for textual programming. This hybrid approach was found to allow for a better retention of computer software university students, when comparing only textual or only visual styles in the learning of programming ([11](#)), showing that it better suits beginner level programmers (a category into which a large percentage of experimentalists fall in).

Behavioural sciences toolboxes

Heron was originally conceptualised to be a framework for creating experiments in the fields of behavioural sciences (e.g. neuroscience, experimental psychology, etc.) and although its philosophy and use cases span a much wider spectrum, its current usage derives from experiments in this field. Other frameworks that specifically target the same fields are pyControl ([12](#)), Bpod (Sanworks LLC, USA) (building on the central design concept of B-control ([22](#))) and Autopilot ([13](#)). Bpod and pyControl are software-dedicated hardware efforts while Autopilot is a software framework that in the same spirit of EPypes, ROS and Heron allows a distributed experimental pipeline albeit restricting the machines to Raspberry Pi computers. All three efforts pay special attention to offering their users tools for creating state machines to define their experiments (each utilising its own way of doing so). Heron currently allows the users to decide if their pipeline would benefit from a state machine design or not and being Python based allows for the use of a plethora of state machine tools in the Python ecosystem ([23](#), [24](#)). This includes the capability to script Nodes that can wrap the Python APIs of pyControl and Bpod (through the Champalimaud Foundation’s pybpod API). This can be of interest to those experimentalists who have invested in the respective hardware modules but would like to expand the capabilities of their pipelines beyond the reach of pyControl or Bpod.

Design benefits

Self-documentation

The Knowledge Graph of Heron immediately offers a succinct overview of the experimental workflow and the dynamics it implements, thus acting also as the primary documentation of the experiment. Armed with a coherent picture of the experiment’s information flow over time, one can access the code of individual Nodes, for a deeper understanding of its details. Grasping the meaning of a few hundred of lines of Python code that most Nodes require to be implemented, one Node at a time, is a much more appealing proposition than opening up a whole code base of a non-Heron experiment and be faced with thousands of lines of obscurely interconnected code arranged in a file system that only makes sense to the developer (and only for a short while after the code’s implementation). Moreover, Python code, in comparison to other lower level languages, helps with readability in a self-documenting fashion (notwithstanding the plethora of in code documentation tools in the Python ecosystem). This self-documentation capability of Heron’s experimental implementations confers obvious benefits to the exchange and reproducibility of experiments and minimises the possibility of misunderstandings when researchers from different groups try to interact with the experiment.

Multiple machines, operating systems and environments

In Heron, each Node runs its own process (practically its own little program, separate from all the programs of the other Nodes). This multi-process approach offers an important competence; running different Nodes on different machines (albeit by taking a hit on system resources vs a multi-threaded approach). This is important since experiments should not be constrained by the Operating System (OS) or the chip architecture that a small part of the experiment might require to run. For example, a fast, high resolution camera might have drivers only for Windows, while raspberry pi cameras can be advantageous since they are easy to multiplex (due to the pi's GPIO and low cost of a raspberry pi with a camera) while online, million parameter, deep learning algorithms will definitely not run on anything other than high spec Linux machines. Heron removes the need to choose between these capabilities since its Nodes (i.e. processes) can run on any machine connected to the main one that runs Heron, through a Secure Shell Protocol (SSH) accessible network connection. When a Knowledge Graph is initiated (i.e. a task is launched), Heron will connect to all the defined machines in the network and will initialise whatever processes it has been directed to start at each of its predefined machines. While the experiment is running, it will take care of message passing between machines and when the Graph is terminated it will make sure all processes are also gracefully terminated. Since Heron uses standard Python to implement most of its Nodes (something that users, as we mentioned above, do not have to adhere to since functionality to work with other code bases exists – see the Rats playing computer games: State machines and non-Python code Nodes example) a Heron experiment can be easily defined on machines with different chip architectures, different OSes and different levels of virtualisation. The general rule is that if a machine with a certain configuration can run the scripts of Python (or other nonPython code) that define the Nodes that need to run on that machine then that script can be part of a Heron experiment and have its Node's inputs and outputs interact with Nodes running on other machines, all set up through Heron's GUI and with minimal user effort. Heron itself (the GUI and underlying communication functionality) has been tested to be operational under Windows (10 and 11, both x64), Linux PC (Ubuntu 20.04.6, x64) MacOS and Raspberry Pi 4 (Debian GNU/Linux 12 -bookworm-, aarch64).

Python and the ease of implementing code

Finally, Python as an implementation language offers Heron another set of desirable (and some not so) consequences. These include the standard pros and cons of Python versus other computer languages. Apart from this, batteries included, Python approach to problems, the main advantage for experiment implementation is Python's extensive community of developers that have contributed to the open source ecosystem of Python libraries for practically any computation imaginable. This extends to also drivers and control APIs for most hardware that an experimenter might require to use. From standard data crunching algorithms to state of the art machine learning ones and from serial communication to drivers controlling high spec equipment there is very little that has not been covered by a Python library. This wealth of ready to go solutions makes the two-tier approach of Heron (design the Knowledge Graph in a GUI and implement the Nodes' behaviour in Python) not just a viable but the preferred approach for any experimental designer. Especially for those experimenters who may not be versed in the latest nuances of low level computer code, but still would like to be fully in control of the behaviour of their experiment. For the cases where a user with deeper knowledge of software engineering has a specific need to use other languages, Heron offers one last benefit arising from the use of its message passing library, OMQ ([14](#)). OMQ is a versatile and easy to use library for passing messages across different processes running on different machines. Most importantly, it includes bindings for almost all commonly used languages. Utilising this library (with minimal effort) a user can create an application in any language that communicates with a Heron Node (practically implementing a small part of Heron's protocol in another application). Then a Python wrapper Heron Node can be

made responsible for the executable's life time. In this way, one has just created, with very little effort, a Node that runs an executable written in some other language but acts just like any other Heron Node passing data to and from any other Nodes.

Methods

Heron's architecture

Node types

Heron defines three different types of Nodes, each implementing a different basic functionality of message passing. Those are the Sources, the Transforms and the Sinks (see the blue, green and red Nodes respectively in the example KG of **Figure 1B** [↗](#)). The Sources are Nodes that generate data (either computationally or by reading them from a device) and thus can only transmit data through the Nodes' outputs. The Transform Nodes can both receive and transmit data through both input and output points and are meant to allow data manipulation. Finally, the Sink Nodes can only receive data and only feature input points. The Sink Nodes are designed to either save data or talk to devices that require only computer input and not input from the external world (e.g. a motor). The Nodes' types only exist to generate a cleaner code base by separating the three types of message manipulation (output only, input then output and input only). There is nothing though (except Python's rule No 7: Readability counts), stopping a user to create side effects of functions implemented by these Nodes other than message passing, thus interacting with machines in ways different to how the Node's type would suggest.

Heron's Actor-based model

Most of Heron's advantages over similar software tools stem from the way it structures the communication between the different Nodes (and thus processes underlying those). Heron's processes do not allow each other to take control of each other and change each other's state. Each process has full control of its state and will only allow another process to influence it through the passing of messages. This is known as the Actor-based model. In contrast, the most commonly used Object Oriented Programming (OOP) model will allow an object to directly change the state of another. For example let's consider the situation where the result of an online analysis on incoming data should be used to change a camera's gain. In an OOP world the object responsible for the analysis would also have to carry a pointer to the memory that represent the object responsible for controlling the camera and directly change the gain (by changing the value of the instance's gain variable) when required. But what happens when another object is introduced latter on that also needs to control the gain of the same camera? What if the change of gain is also dependent on the gain's history? Who is responsible for correctly changing this parameter when there are more than one objects vying for control and will the introduction of the new object require changing the code on both the camera and the analysis object code bases? The Actor-based model solves these problems by allowing the camera's gain to be changed only by the object that controls the camera itself. All other object can only request such a change by sending request messages to the camera object (in Heron's case the camera Node). In this way, when a user composes a Node they have to think only about what that Node does and how it communicates with other Nodes, and never about the way code outside it might change its behaviour (which Heron with its Actor-based model will never allow).

Heron GUI's multiple uses

To understand Heron's code structure, one must initially appreciate its dual role in designing and running an experiment. When a Graph (short for Knowledge Graph) is not running, Heron acts as a Graph designer, offering a GUI where a user can create and delete Nodes, connect them with

Links and assign values to their parameters. During the design period, only one process is active, the one running the Heron GUI (Editor, see [Figure 2](#)). When, on the other hand, a Knowledge Graph is running, the Heron process stops being a Graph design application and assumes the role of a director in an actor based model ([2](#)). It can then concurrently compute and run a GUI for the experiment where the user can update the parameters of the different Nodes on the fly (as an experimental Control Panel). In this actor-based model each Node is represented by two processes (Worker and Communication, see [Figure 2](#)) while there are three more processes acting as message forwarders between all other processes (Proof of life, Parameters and Data). That means, a running experiment is constituted by $(\text{Number of Nodes}) \times 2 + 4$ processes (see [Figure 2](#)). Each process is an actor that can receive and transmit messages, make local decisions (i.e. decisions that can effect only itself) and determine how to respond to incoming messages. In the (most common) cases where the Nodes running are all implemented by Python code then the Heron process is responsible for initiating the three forwarders and the com processes for all the Nodes. Each com process will then initiate the corresponding worker process. In the special case (see Rats playing computer games: State machines and nonPython code Nodes example) where a Node will call an executable instead of a Python script then the worker process can also be responsible for initialising (and terminating) the executable's process. As mentioned above, each Node is represented by two processes. In the code those are called the com and worker process. The worker process is the one that runs the Node's script defined by the user. The com process is responsible for i) grabbing messages that come out of other Nodes and are meant to reach the Node (as defined by a Link between two Nodes in the Node Editor), ii) passing those messages to the worker process, iii) receiving any messages the worker process has to pass to other Nodes and iv) passing those messages to the com processes of all the Nodes that should receive them. The passing of messages between com processes of different Nodes is facilitated by the Data Forwarder process. The worker processes also communicate directly with the Heron process through two separate forwarders. The Parameters forwarder is responsible for passing to the worker processes the parameter values assigned by the user to the processes' respective Nodes on their GUI. This allows the user to also manipulate the state of each Node while an experiment is running. Through this functionality the Heron GUI becomes (while a Graph is running) also a control center through which an experimenter can interact with the experiment by changing live the Nodes' parameters. The Proof-of-life forwarder is responsible for receiving an initial message from the worker process which is then passed to the GUI process letting it know the worker is up and running.

As mentioned above, Heron allows any of the Nodes in a Graph to be initiated and executed in machines different to the one running the main Heron process. At the level of the processes, that means that Heron, if instructed to run a Node on a different machine, will run only the worker process of that Node on the different machine while its com process will run on the same machine as Heron. That has as a drawback that a user cannot put multiple Nodes on a separate machine and expect them to interact (through messages) within that machine since all message passing happens through the com processes which will always run on the Heron running machine. Future version of Heron will address this limitation by allowing Heron to run Graphs headless (without the GUI process being active) which will allow sub-Graphs to fully run on one machine and communicate their result to Nodes in the machine running Heron.

Code architecture

Heron's code is separated into three main folders each pertaining to one of the three aspects of its basic functionality (see Supplementary Figure 1 for all of Heron's folder structures). The Communications folder includes scripts that deal with the low-level communication between all Nodes that make up a running Graph (experiment). The GUI folder holds scripts that deal with Heron's Graphical User Interface (GUI). The Operations folder keeps all the code that defines all the Nodes that Heron recognises and can use to create a Graph with. The Operations folder is further subdivided into the Source, Transform and Sink folders, each holding codes according to the type of the Node it implements. The Operations folder also holds symbolic links to code

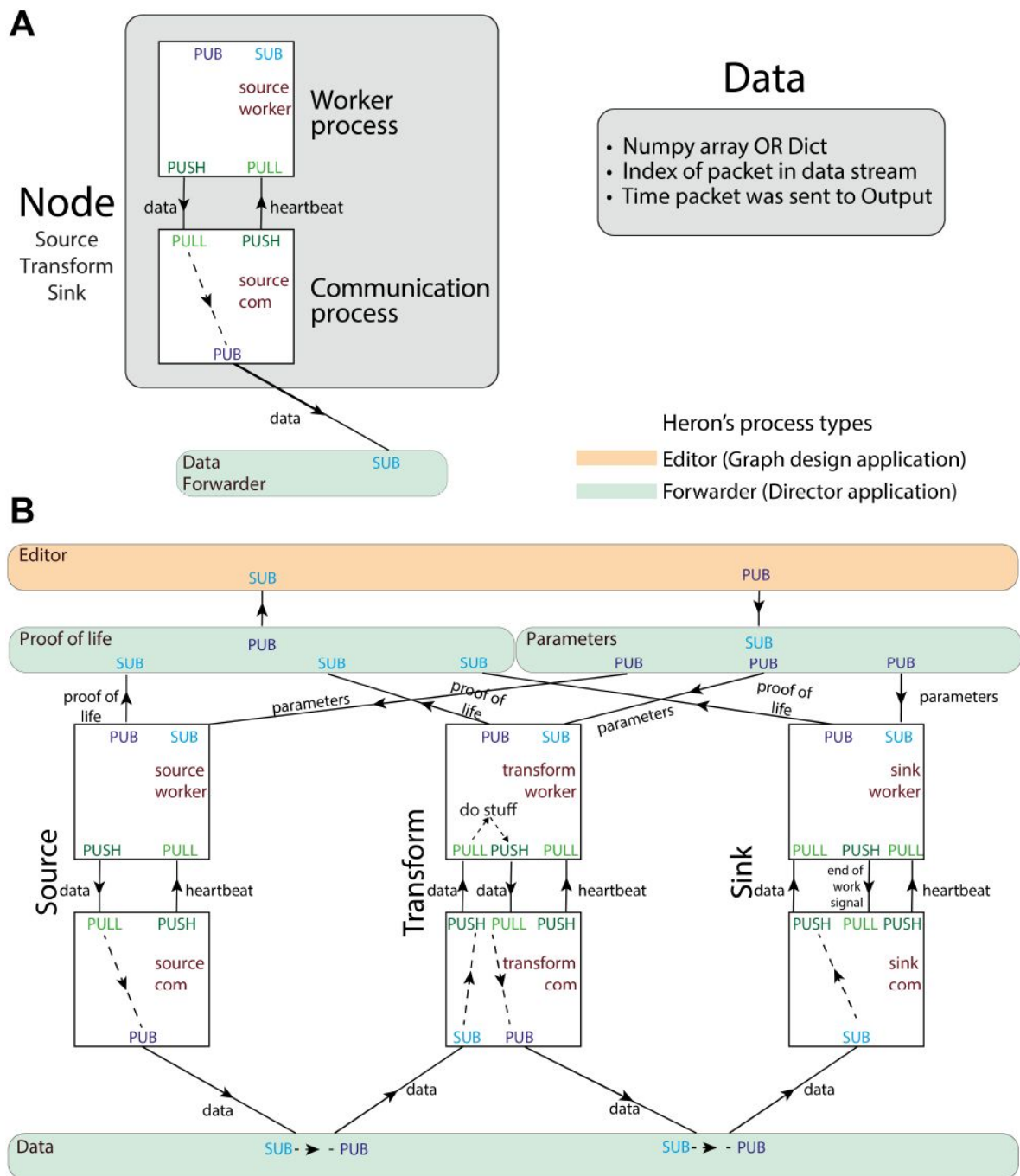


Fig. 2.

Heron's design principles. A. On the left, Heron's Node structure showing the two processes that define a running Node and how messages are passed between them. The design of the special message defined as 'data' is shown on the right. B. Heron's processes and message passing diagram. Each rectangle represents a process. The top orange rectangle is the Heron's Editor process, while the other three green rectangles are the three Forwarders. The Proof of life forwarder deals with messages relating with whether a process has initialised correctly. The parameters forwarder deals with messages that pass the different parameter values from the editor process (the GUI) to the individual Nodes. Finally, the data forwarder deals with the passing of the data messages between the com processes of the Nodes. The squares represent the worker (top) and com (bottom) process of three Nodes (a Source, a Transform and a Sink from left to right). Solid arrows represent message passing between processes using either the PUB SUB or the PUSH PULL type of sockets as defined in the 0MQ protocol. Dashed line arrows represent data passing within a single process.

repositories separate from Heron. Those, assuming a specific folder structure, are recognised by Heron as valid Node describing codes. The scripts in the Communications folder are class definitions for 8 objects: 6 for the worker and com objects for each Node type which implement Heron's communication protocol, one for the object that deals with the network connectivity (SSHCom) and one re-implementing pyzmq's (Python's 0MQ bindings) Socket object, adding to it the ability to pass numpy arrays and dictionaries as messages. This is required because in Heron all messages are either numpy arrays or python dictionaries. That means that the worker functions of the worker scripts of the Source and Transform Nodes will always return either a list of numpy arrays or a list of dictionaries. Each element of the list corresponds to one output of the Node. The Operations folder has three levels of subdivision. Immediately bellow it are the Source, Transform and Sink folders and inside those are folders representing groups of Nodes in each Node type (e.g. Vision for Nodes that have to do with computer vision). Inside those subcategory folders are the folders that hold the scripts for each Node (e.g. Camera which holds the scripts that read a web camera into Heron). Inside each Node's folder there are a minimum of two scripts with name suffixes `_com` and `_worker`. The `_com` script allows the user to define a Node's characteristics (parameters, inputs and outputs) with a few lines of code and without requiring any GUI relevant code (Heron takes care of that). The `_worker` script is responsible for the functionality of the Node (being the script that is run by the Node's worker process through the `node_type_worker` object) and implements a minimum of three functions. These are the initialisation, the worker and the end-of-life functions (the names are arbitrary and the user can define them as they please). The initialisation function is run when a Node is first started by Heron (i.e. its com process is up and running and its worker process has just started but is being tested before it starts receiving and transmitting data). The worker function is the main function that implements what the Node is supposed to do. The worker functions of the three types of Nodes are implemented differently. In the case of a Source Node, the worker function needs to be an infinite loop that somehow generates data and passes them on the Node's com process at the end of every loop. The Transform and Sink Nodes need a worker function implemented as a callback since their worker processes will call the worker function every time there is any data arriving at the input of the Nodes (i.e. any time their com process has received a message from another com process and has passed this to its worker process). Both the Transform and the Sink Nodes will stop accepting messages until their worker functions have returned and Heron is designed to have no message buffering, thus automatically dropping any messages that come into a Node's inputs while the Node's worker function is still running. Finally, the end-of-life function will be called when a worker process hasn't received a heartbeat signal from the source process for a pre-determined amount of time and its role is to gracefully terminate the process.

Usage

There are two skills that a user should possess in order to aptly use Heron. Firstly, one requires a familiarity with Heron's GUI which allows i) downloading and installing new Nodes from existing repositories, ii) defining a local network of computers on which the different Nodes can run, iii) setting up a pipeline using the existing Nodes and iv) running the pipeline all the while being comfortable in debugging it as problems arise. The second skill is the implementation of new Nodes based on the user's individual needs. In this section we will provide a basic description of both the GUI usage and the development of new Nodes.

Using Heron's GUI

Adding new Nodes from pre-existing code

Heron comes pre-packaged with a small set of Nodes that have a generic enough usage that most users would find useful. An important point though about Heron is that every user will be developing their own Nodes which in most cases will take the form of code shared in some online repository. Heron is designed to easily access repos that have been developed following a specific file structure to represent a set of Heron Nodes and integrate them into its GUI and workflow

without the user needing to do anything else other than create/download the repository and point Heron to it. This also simplifies the further development of Nodes by the community of users since a new Node repository does not have to interact with the main repository of Heron and thus avoids all the pitfalls of pushing, pulling and merging code repositories at different levels of maturity.

Local Network

Heron's GUI allows an easy definition of the local access network (LAN) of machines that will run Nodes forming a single pipeline. A user has only to provide the IP, port, user name and password of a machine in the LAN and Heron will communicate between machines using an SSH protocol, taking care of issues like process lifecycle on different machines, opening and closing ports and proper passing of messaging between processes over the network.

Setting up a pipeline

Once all the Nodes' repositories have been made known to Heron and the LAN of all machines has been set up, a user needs to implement the experiment's pipeline. This is achieved again graphically by introducing the required Nodes in the Node Editor (main window of Heron), setting up their parameter values and finally connecting the Nodes together by creating links between outputs and inputs. Heron allows many to many connectivity, meaning a Node's output can connect to any number of inputs and an input can receive any number of outputs.

Running a pipeline

Once a pipeline has been defined (generating the Knowledge Graph of the experiment) then running it is achieved by pressing the Start Graph button of Heron. Heron will go through each Node (in order of addition to the Node Editor) and will start the processes that the Node represents (see Heron's Architecture for more details). It will then connect all the processes with OMQ sockets as defined by the links between the Nodes and pass the Nodes' parameters to the worker process of each Node. The pipeline of data being generated by the Source Nodes, being transformed by the Transform Nodes and finally saved or turned into control of machines by the Sink Nodes will keep on running until the user pressed the End Graph button. At this point Heron will gracefully terminate all processes (including the ones running on separate machines) and close down all communication sockets.

Creating a new Node

Heron users will develop their own Nodes for their specific experiments. To facilitate this, Heron provides a series of tools. First there is a set of templates that offer a scaffold on top of which a user can build their own code. The templates have the required code elements that all valid Heron Nodes must possess and are fully documented to help a user quickly build functioning Nodes. An abbreviated and annotated Transform template for the com and worker scripts can be seen in Supplementary Box 1A and 1B. On top of Node templates, Heron provides a GUI based system for generating new Nodes. This will create the required files and all the common between Nodes, skeleton code, appropriate to the type of Node under development and using names (for functions and variables) chosen by the user during the GUI based creation process. Finally, in order to expedite the correct placement of new Nodes in pipelines Heron offers a Node (User Defined Function 1I 1O) designed to allow python functions to be run as Nodes in Heron (without the user having to follow Heron's API). This Node can be used to simulate inputs and outputs to an under development Node in order to test and debug its functionality before incorporating it into a full pipeline.

Node Repositories

As described above, Heron offers the tools to integrate any new code (designed with the correct file structure) into its collection of Nodes and make it available in its GUI. Although not necessary, good practice would be to develop any new Node (or closely related group of Nodes) as part of a separate repository so that the Node can be easily shared with the rest of the community. Currently a public GitHub organisation called (rather unimaginatively) *Heron-Repositories* [🔗](#) is hosting both the main Heron Git repository and all other Git repositories of Nodes developed to cover the developers' experimental needs. Any of the individual Heron Nodes repositories can serve as an easy to follow example on the file structure expected by Heron for successful integration of new code. All of the Nodes presented in the Results paragraphs examples can be found in this repository.

Results

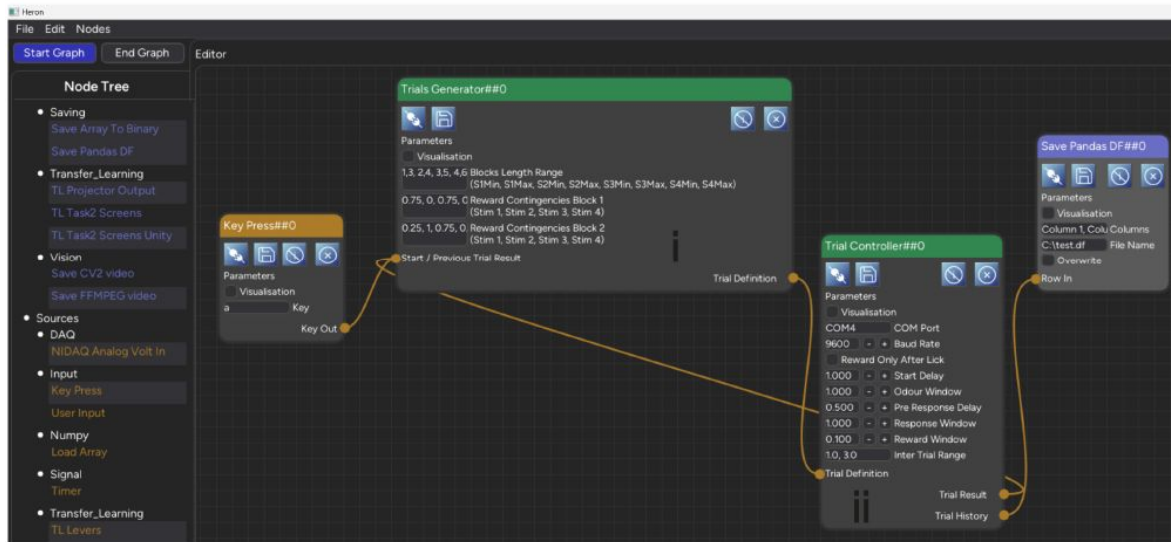
Here we showcase a number of experiments implemented in Heron. Each example has been chosen to highlight one of Heron's competences as described in the Design benefits. As mentioned above, all the Nodes used to construct the examples presented here can be found in the Heron-Repositories GitHub organisation. We are not making public the specific experiment files since these are hardware specific and would need large changes to be made compatible to any other hardware. But Heron's graphical nature makes it easy to go from an image capture of an experiment's Heron GUI (see **Figs. 3** [🔗](#) to **5** [🔗](#)) to a working experiment by simply combining the required Nodes.

Probabilistic reversal learning.

Implementation as self-documentation

The first experimental example is provided here to showcase how Heron's implementation of an experiment becomes the easiest way for non-developers to acquaint themselves with the experiment and its logic. Thus, here we describe the implementation from the point of view of someone who sees it for the first time and is trying to understand what the experiment is (without accessing any other publication or written explanation). As seen in **Figure 3** [🔗](#), this experimental pipeline is made up of 4 Nodes. One is called a "KeyPress" which since it connects to an input named 'Start / Previous Trial Result' seems to play the role of the start button of the experiment. The last Node is a "Save Pandas DF" which suggests saving the output of the 3rd Node (the "Trial Controller") in a row of a pandas DataFrame. The main experiment seems to be defined by 2 Nodes, one named "Trial Generator" and one named "Trial Controller". We can immediately conceptualise the pipeline as a two part one, where the first part generates some kind of trial state which it then outputs as its 'Trial Definition' output to the second Node which inputs that 'Trial Definition' and runs (Controls) that trial given its state. We notice that these two Nodes are reciprocally connected, meaning that the "Trial Generator" requires the output of the "Trial Controller" (named 'Trial History') to generate the definition of the next trial. Looking a bit more closely at the names of the parameters of the Nodes we can deduce a number of things about the experiment's structure and function. From the Trial Generator parameters, we see that trials seem to fall into two blocks. We can also see that the experiment has trials with 4 types of Stim (maybe short for Stimulation, or Stimuli). We can assume (but would need to verify from the code) that the Reward Block Contingencies mentioned for the two Blocks are the probabilities of a reward given the type of Stimulation. So, we have surmised that the trials come into blocks of specific length (probably a random variable drawn from some distribution from the user provided Blocks Length Range parameter) and of specific trial type (one of four) and each trial type in each block has a user defined reward probability. So far so intelligible. By looking at the Trial Controller Node's

A



B

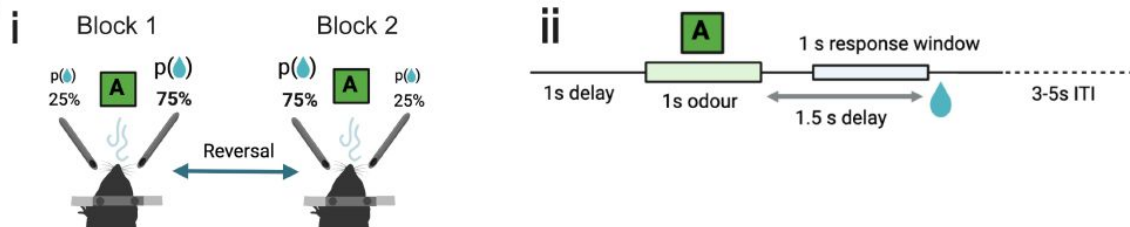


Fig. 3.

A. The Knowledge Graph of the Probabilistic Reversal Learning experiment showing 4 Nodes comprising the task, two of which (Trial Generator and Trial Controller) are connected in a loop (the output of one is fed into the input of the other). B. The schematic or mental schema of the experiment. Notice the direct correspondence (i and ii) between the Heron Nodes and the two main components of the experiment's mental schema as well as the Node's parameters and the schema components' basic variables.

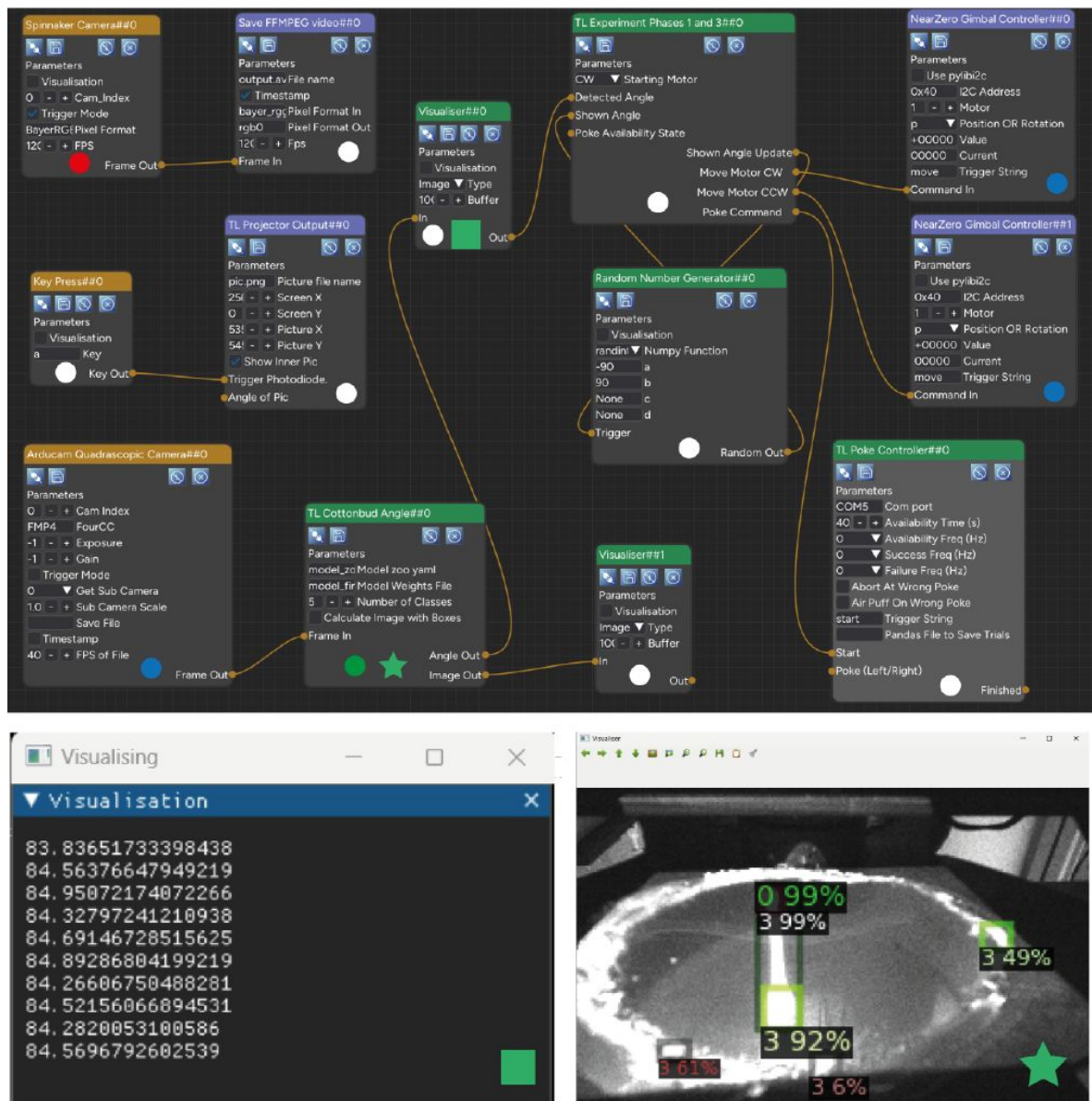
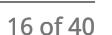


Fig. 4.

The Knowledge Graph of the Fetching the cotton bud experiment. The coloured circles at the bottom middle of each Node and the coloured star and square (where shown) are not part of the Heron GUI but are used in the Figure to indicate in which Machine/OS/Python configuration each Node is running its worker script (circles) and which visualisation image at the bottom corresponds to which Node. For the circle the colour code is: White = PC/Windows 11/Python 3.9, Red = PC/Windows 11/ Python 3.8, Blue = Nvidia Jetson Nano/Ubuntu Linux/Python 3.9, Green = PC/Ubuntu Linux in WSL/Python 3.9. The two smaller windows below Heron's GUI are visualisations created by the two Visualiser Nodes. The right visualisation (coming from Node Visualiser##1) is the output of the Detectron2 algorithm showing how well it is detecting the whole cotton bud (detection box 0) and the cotton bud's tips (detection boxes 3). The left visualisation box (output of the Visualiser##0 Node) is showing the angle of the cotton bud (in an arbitrary frame of reference where the 0 degrees would be almost horizontal on the screen and the 90 almost vertical). This angle is calculated in the TL Cottonbud Angle Node which is responsible for running the deep learning algorithm and using its inference results calculating the angle of the cotton bud. As shown the TL Cottonbud Angle Node is running on a Linux virtual machine (since Detectron 2 cannot run on Windows machines).



parameters we see first of all that the Node requires a COM Port and Baud Rate to be defined showing that it is controlling some device through a serial port. The ‘Reward Only After Lick’ parameter tells us that this is an experiment where the subject needs to lick (and in some cases this is the only way to get a reward). The names of the rest of the parameters indicate that the experiment is an olfactory one (see ‘Odour Window’) where the subject gets to experience an odour (maybe one of the four stimulations described in the Trial Generator), then gets a pre-response delay, then a response window time and finally a reward window. In order to get a better picture of what the experiment is actually doing we next look at the code. But most importantly, we know exactly where in the code base we should be looking without needing to search through tens of files to follow an obscure logic to figure out where the bits of the code that implement the actual logic are. There are two Nodes of interest so we should, as a first step, look at the worker scripts of those two Nodes. The Trial Generator is an undaunting 127 lines of code while the Trial Controller is 251 (empty lines included). Yet we can zoom even more in where we should initially look by simply ignoring the initialisation and *end_of_life* functions and just concentrate on the *worker_function* of each script. Those two functions are shown in the *Worker_function* Code in Supplementary Boxes 2A and 2B. We need only two pieces of prior knowledge to understand the code. One is that the worker functions of the Transform (and Sink) Nodes are call-backs that are called every time the Node receives a message to any of its inputs. Both the Trial Generator and the Trial Controller are Transforms (they have both inputs and outputs and are marked as green on the GUI). The second is that each Node outputs through the return statement of its worker function a list of numpy arrays with each element of the list being the array that is outputted by a specific output of the Node (in order from left to right in the list corresponding from top to bottom in the Node). So, a Node with two outputs will have a worker function that returns a list with two numpy arrays / dictionaries. In this case, for example, the Trial Controller’s worker function should return a list with two arrays i.e. the Trial Result output and the Trial History one. With this in mind and looking at the Trial Generator we can see that it takes in the previous trial’s stimulation number and if the correct port was licked, use this information to keep a running track of the correct licks for each port, decide which block we currently are in, decide which stimulation to generate next and offer or not a reward. By following the Trial Controller *worker_function*’s code we can easily validate our prior speculation about the temporal structure of a single trial following a path of delays interspersed with stimulation and potential reward. An overview of the actual experiment running on Heron, also showcasing how Heron communicates with the Arduino board that does the hardware control, can also be seen in Supplementary Figure 2, together with a schematic of the experimental timeline.

Fetching. Four environments, three Operating Systems, two machines, one pipeline

This next experiment showcases Heron’s ability to run Nodes whose worker process runs on machines separate from the one running the Heron GUI. The experiment itself requires the monitoring of a rodent as it tries to fetch a cotton bud from a random point in a large arena and drop it into a shallow pot under a hole at one of its corners. The hardware setup has a Grasshopper3 USB 3.0 colour camera (GS3-U3-23S6C-C, FLIR) which records the whole of the arena from the top at 120 frames per second with HD resolution, while 4 smaller, black and white, 30 frames per second cameras (OV9281, ArduCam) are used to capture the animal from 3 different angles and to also monitor the target pot from underneath (since its base is transparent plastic) using the fourth camera. The experiment needs to also know when the animal has deposited the cotton bud in the hole and also record at which angle this has happened. When the animal has deposited the cotton bud it is rewarded with a treat from a nearby reward port. The pot rotates removing the cotton bud from the arena (whilst another, empty pot takes its place) and the cotton bud is thrown back into the arena for the animal to fetch again. The Grasshopper camera at the time of the experiment’s development could run only on Python 3.8. The 4 smaller cameras are part of a system that synchronises them, providing a stitched up single frame of 5120×800 pixels resolution, (ArduCam 1MP*4 Quadrascopic Monochrome Camera Kit) which can run only on

either a Raspberry Pi (>=3) (Raspberry Pi Foundation), a Jetson Nano (Nvidia) or a Xavier (Nvidia) single board computers. The angle detection of the cotton bud when it is in the pot is done using Meta's Detectron 2 (15 [🔗](#)), a deep learning algorithm which we trained with a few hundred samples to detect the cotton bud's edges when in the pot using as an input the part of the OV9281 camera frames that come from the camera underneath the pot. Detectron 2 requires an Intel or AMD CPU based computer running Linux. Heron itself and all the other Nodes for this experiment run on a Windows PC with Python 3.9. The choice to run the Heron GUI on the Windows machine was made to keep the latency of the 120fps camera to a minimum. So, the experimental pipeline needs the following Machine/OS/Python configurations: Intel/Windows/Python 3.9+, Intel/Windows/Python 3.8, Intel/Linux/Python 3.9+ and ARM/Linux/Python 3.9+. To create all the above configurations, we first made in the main Windows 11 machine (that Heron runs on in a Python 3.9 environment) a separate (conda) environment with Python 3.8 and the required Spinnaker (for the Flir camera) python package. Then we set up on the same machine a Windows Linux Subsystem (WLS 2) virtual machine, running Ubuntu and Python 3.9 with all the required packages to run Detectron 2. Finally, we connected the Windows machine through a LAN to an NVIDIA Jetson Nano with the Arducam Quadrascope system running Ubuntu and Python 3.9 with all the packages for the Arducam system to operate. To summarise, the pipeline (which can be seen in **Figure 4** [🔗](#)) when run, is utilising two physically separate machines, three operating systems (one Windows 11 and two Linux, one on a virtual machine) and four different Python environments. Once each machine/OS/Python environment is up and running and each one can run the Nodes(s) that it is supposed to run by itself (something that can be tested and debugged at the level of an individual Node without requiring the whole pipeline to be up and running), then assembling the pipeline is as simple as connecting the Nodes appropriately on Heron's Node Editor and telling each one (through the Node's secondary parameters window) which computer it should run on and which python executable it should call to run the worker script. Heron hides from the user all of the work required for the different processes in all the machines to start at the right time, connect correctly to each other, exchange data while the pipeline is running and finally gracefully stop when the pipeline stops without leaving hanging processes and inaccessible bits of memory all over the place. An overview of which machine runs what Node can be seen in **Figure 4** [🔗](#) while Supplementary Figure 3 shows a snapshot of an animal having fetched a cotton bud while its angle is being live detected by one of the Detectron 2 algorithms.

Rats playing computer games

The rats playing computer games experiment teaches a rat to rotate a line on a screen using a left and a right lever press to hit a target line and avoid a trap line. It presents a rat with a nose poke hole, two levers (left and right to the hole) and two screens to the animal's front and right, when it is poking. At the final stage of training the animal should be able to first nose poke, look at the screen at a set of jagged lines and press one of the two levers to make one of the lines rotate toward the correct line (target) and away from the wrong one (trap). Once this has been achieved the jagged lines disappear and a separate visual cue appears (usually animated) letting the animal know that there is reward in the reward port (see Supplementary Figure 4). This is a very challenging behaviour for a rat and the experiment needs several stages of shaping to teach it to the animals. Each stage of shaping toward the final behaviour requires a set of different visual stimuli and a different set of states in a rather large state machine. Also, at the conception of the experiment there was no prior experience on the ideal path to the final behaviour, so a very malleable stimulus generation technique and state machine development was required in order for a large number of ideas to be tested in a small amount of time. Because of the above this experiment is ideal in showcasing a series of Heron's capabilities. These are Heron's ability to work with non-Python code, Heron's workload management, the tools Heron provides to keep track of complicated experiments and record all required information and finally Heron GUI's capability to act not just as a Graph designer but also as a control panel while a Graph is running.

State machines and non-Python code Nodes

As mentioned above, although Heron's code base for Node development is in Python, it is relatively easy to create Nodes with code bases on different languages. One way to do this, (which is the way most scientific computing Python libraries are using), is to use Python interop technologies which allow functions written in other languages (most commonly languages of the C family) to be called by Python. Although this is very powerful it is a painstaking task, can be very time consuming, and usually requires significant experience with both Python and whatever other language one is working with. Another, not as a low-level approach, offering less control but faster implementation times, is the use of the 0MQ library to pass messages between a Heron Node and an executable written in another language (what Heron is doing to connect all of the Nodes of a pipeline but with a significantly toned-down communication protocol). Here we took the second route. In the Unity game engine (Unity Technologies, US) we made a simple 2D game using C# code, that covered all the possible visual stimuli we needed to show to the animals at any stage (see [Figure 5D](#) for a snapshot of the Unity development environment). Instead of using the standard game inputs (e.g. keyboard or game controller) to control what and when was played on the screen, we used a string of commands that was received by the game executable through a 0MQ SUB socket (using NetMQ, one of many C# 0MQ bindings). The Unity generated executable was also designed to do a handshake through a PUSH/PULL socket with whatever process initiated it. This made sure the initialising process knew if anything was wrong and also was able to send some initialisation messages. Once the game executable was ready we created a standard Heron Sink Node whose initialisation function would start the executable, handshake with it and send it the Node's parameters (e.g. if the game was meant to show its stimuli on one or two screens). Then its worker function would just pass to the correct socket any string that would be received at its input from the other Heron Nodes. Properly formatted strings would be understood by the game and update its state accordingly. The Node's *end_of_life* function would finally close down the executable when Heron's Knowledge Graph was terminated. This use of process control done not directly from the main Heron process but from a Node's worker process is compatible with the concept of Heron as an actor-based framework where an actor (a Node's worker process) can also initialise and end other actors (in this case the Unity executable). Regarding the state machines used in this experiment, we found that no matter how large and complicated a state machine gets, the difficulty lies in its initial design and not in its implementation. Here we used *statemachine* ([16](#)), a python library that allows the definition of a state machine and its individual states with easily attachable callbacks at state transitions.

CPU load

Using multiple processes rather than multiple threads does incur a hit in the total CPU load Heron will require in order to execute a Graph. The 'Rats playing computer games' experiment, with a 15 node Graph (the largest one we had ever had to execute over multiple experiments implemented in Heron) offers a good test bed to showcase Heron's CPU requirements. The experiment was conducted on a Windows 10 x64 machine with an 8 core i7-7700K CPU clocking at 4.20GHz. The Graph runs an acquisition and save of a 120fps 576×676 pixels video, an acquisition and save of a 20KHz, 5 channel DAQ trace updated at 10Hz, a Unity game running concurrently on two HD screens updating at 120Hz and an acquisition and save of a 30fps 5460×800 pixels video running on the Jetson Nano machine. When the computer was idle it showed a base CPU load of just around 10-15%. When Heron was up but not running a Knowledge Graph, it consumed about 2-5% of the CPU for a total load of 15-20%. With the above experiment running, the CPU load hit 100% with the Unity game requiring around 24% and the remaining 45-55% used by the Graph's 32 processes (one process was running on an NVIDIA Jetson Nano machine). The above numbers though are true if the operating system was allowed to assign CPU cores to the different Heron operations as it saw fit. Heron has a feature which allows the user to force the operating system to keep a worker process running on a specific CPU core. Rerunning the above Graph while setting 10 of the 15 worker processes to specific CPUs and allowing the system to optimise the load for only 5

of them (the Spinnaker Camera, Save FFMPEG Video, Arducam Quadrascope Camera, Save Array To Binary and TL Taks2 Screens Unity) dropped the total CPU load to 80%, again with the Unity game consuming around 24% while all Graph python processes consuming around 25%. The above specifications show that Heron is capable of running Graphs with heavy acquisition and save loads. Also, Heron's ability to utilise multiple machines in a single Graph means that processes requiring exceptional CPU, memory and / or GPU loads can be offloaded to machines other than the one running the Heron GUI. In the above example this is the case with the acquisition and save of the Arducam video. Also, before implementing the CPU setting feature we used to run the Unity game on a 3rd machine in order to keep the 24% CPU load requirement out of the main Heron machine. We stopped using this 3rd machine once the CPU setting feature was in place, and the transition from three to two machines required only a single line change in the Node's secondary window to indicate where the worker script was to be found.

Synchronisation (time alignment)

Multiple processes, running over different machines can become extremely difficult to synchronise, especially to the tight requirements of a scientific experiment. Heron offers a set of tools to allow users to time align every single packet of data generated and saved by any Node to any other packet irrespective of whether the packets were generated by Nodes on the same or separate machines. Here we use the 'Rats playing computer games' experiment to showcase how such time alignment would be achieved between a few of the experiment's Nodes.

More specifically the experiment has two separate video captures which, since they happen on different machines, they can only be synchronised through an external clock. This is similar to an electrophysiology data acquisition system whose data points can be synchronised to other data sets (e.g videos, or button presses), only by an external clock (a TTL pulse generator usually) that is simultaneously recorded by all devices. The whole setup is shown diagrammatically in **Figure 5C**.

We use a 120fps projector (whose output is controlled by the Node TL Projector Output##0) to act as our base clock. By pressing a key after the Graph has started running, a part of the projector's output (depicted as a screen in **Figure 5C**) turns blue and thus stimulates a blue sensitive photodiode. This happens at every frame thus the photodiode generates a 120Hz TTL pulse train timed to the onset of the blue pass of the projector. That train starts when the key is pressed and stops when it is pressed a second time (thus demarcating the start and stop of the recording part of the session). The photodiode's TTL pulse acts as an external trigger to the Spinnaker (FLIR) camera whose output is being captured by Heron's Spinnaker Camera##0 Node. The same pulse is also fed to a National Instruments USB Data Acquisition System (NIDAQ) and to an Arduino. The NIDAQ is being captured (at 10 packets of 2000 points each per second) by Heron's NIDAQ Analog Volt In##0 Node. The Arduino runs a small loop that generates a second TTL pulse for every 4 TTL pulses coming in from the Spinnaker camera (thus creating a 30Hz TTL train). This TTL train is fed as an external trigger to the four cameras of the Arducam system as well as to the GPIO of the Jetson Nano machine that is running the Heron Node which captures the Arducam's video frames output (Arducam Quadrascope Camera ##0). The Arducam system generates a TTL pulse every time the global shutters of the four cameras fire and that is fed into the same NIDAQ as the Spinnaker TTL pulse out (see left column of **Figure 5C**).

The Spinnaker camera at every trigger generates not only a frame but also the frame's id. This is captured by Heron's Spinnaker Node's Worker process and saved to its Substate dataframe together with the equivalent id of the Node's data packet and the timestamp of this (see **Figure 5C** "Spinnaker Camera Worker Process Data" table). A Substate dataframe is Heron's way to allow the designer of a Node to save information (to a pandas dataframe) every time the Worker process runs a pass through its loop. This is done through Heron's API requiring only a single line of code with the information to be saved to the dataframe.

Any failures of the Node to capture a frame get registered in this correlation between camera ids and packet ids. The Spinnaker Node sends the captured frame to the SaveFFMPEGvideo##0 where an ffmpeg pipeline saves each frame to a video stream. As mentioned before, Heron does not offer any capability to buffer packets, so if the FFMPEG Node is too late to save any frame then the next one gets lost. Heron records this in the FFMPEG Node's Com process log file. A Com processes log file (optionally set for any Node by filling in the log file's path to the Node's secondary window appropriate entry) records the id of any packet that arrives from an upstream Node as given by that Node and the id of that packet as given by the current Node. In this way any dropped packets are registered and a one to one equivalence between packets sent and packets received can always be established. In this case, the FFMPEG Node's Com process log file indicates the id of each packet that has left the Spinnaker Node and arrived in the FFMPEG Node and the id assigned to it by the FFMPEG Node. The id of the FFMPEG Node corresponds to the frame id of the video generated by the ffmpeg pipeline in the Node. In this way we can now correlate any frame of the produced video to the id of that frame generated by the Spinnaker camera despite some dropped frames in the process (in the shown example the camera generated a total of 216300 frames and the final video had a total of 216244 frames, a drop of 56 frames dropped over the two Nodes, i.e. a 0.02%).

At the same time the Arducam system (triggered by the subsampled TTL pulse from the Spinnaker camera) generates frames at a rate of 30fps. These are captured and saved by the Arducam Node running on the Jetson Nano. The Node is also registering (for every incoming frame) the equivalent id of the TTL pulse that generated that frame by also reading the Jetson Nano's GPIO port. In this way the Arducam Node's Worker process can generate a Substate dataframe that correlates the packet's id as given by the process (and thus the id of the video's frame) with the TTL pulse id that produced that frame.

In order to now correlate the frames of the Arducam video with those of the Spinnaker video we use the saved TTL pulses from the NIDAQ instrument as saved by the Heron NIDAQ Node. The id of the frame captured by the Spinnaker camera corresponds to the number of the pulse in the Spinnaker pulse train and the id of the pulse saved in the Jetson Nano's GPIO port corresponds to the number of the pulse in the Arducam pulse train. By knowing which Spinnaker pulse has generated which Arducam pulse one can work backwards through the above described correlations all the way to knowing which frame in the Spinnaker video corresponds to which frame in the Arducam one despite those being captured on different devices and some frames getting lost by Heron's "dropping of packets" architecture. In [Figure 5C](#) we show more specifically how the last pulses of the TTL trains in the NIDAQ can be used to trace the correspondence between the last frames in the videos of the Spinnaker and Arducam systems using Heron's Substate dataframes and log files.

On the fly experimental control

As mentioned above, the "Rats playing computer games" experiment is a rather demanding task that requires slow and careful shaping on the subjects' behaviour. Certain parts of the training could not be automated (especially when we were trying ideas that we didn't not know if they were relevant to the final shaping protocol). During those sessions the experimenter had to change a number of the experiment's parameters on the fly as the subjects produced behaviour given both how well they were doing and how the experimenter perceived certain aspects of their behaviour that were too hard to quantify (and thus use in an automated system). Examples of these experimental parameters were the number of reward pellets offered to the subjects, the maximum angle between the moving line and the target line and the speed of the moving line (and thus the amount of time required for a lever to be kept pressed), the amount of time the reward was available after a successful trial and whether the levers would vibrate or not when pressed correctly, etc. (see [Figure 5E](#)). Heron allowed us to operate in this "experimenter in the loop" fashion because it allows the Worker process of any Node to update its parameters as they are changed by the experimenter through the GUI while the Graph is running. Which parameters

are capable of online updating is up to the author of the Node and Heron allows both updatable and non-updatable parameters (which are easily defined in the Worker process code through Heron's API).

Discussion

We have presented Heron, a new tool for coding up experimental pipelines. We have put forward the proposition that using Heron instead of the many other frameworks that one can utilise to create software to run experiments has a series of advantages. It can practically self-document, creating Knowledge Graphs (KG) that are as close as possible to one's mental schema of the experiment. These KGs and code bases are easy to follow by researchers other than the developers of the experiment, irrespective of the complexity of the experiment. It can trivially connect processes that run on different operating systems and machines in a single, unified pipeline. For example a series of raspberry pi computers, each reading some cameras or other sensors, can connect and pass the data to a Linux based, many GPU, machine that does online machine learning analysis while these results can pass to a PC machine running a computer game controlled by those results. It is using a language (Python) for the development of experiments that is one of the easiest and most versatile computer languages with a large community of developers and rich libraries for most functionalities. Finally, it is versatile enough to allow easy integration of code bases written with languages other than the one it has been developed in. We are arguing here that Heron's learning curve, starting from a basic capability in Python, is measured in the few hours of trying to create a couple of new Nodes and joining those together in order to create some toy experiment. Once that is understood then the limit to what can be achieved is defined by the level of Python knowledge of the developer. Heron has been conceptualised to grow into a community project. Both itself and the repositories holding extra Heron Nodes are open source under an MIT licence. The separation of repositories that hold the main Heron code and the individual Nodes' code bases allows for the growth of a Node ecosystem where users will be able to share their development using standard repository based tools. Finally, the developers welcome efforts for collaboration with the aim for Heron to eventually become a multi-developer, collaborative project expanding with capabilities covering the needs of experimental scientists in all experimental fields and beyond. The code for all repositories used in this report can be found at <https://github.com/Heron-Repositories>, while Heron's documentation is hosted in <https://heron-42ad.readthedocs.io/en/latest/>. Heron's name is a tribute to one of the first known creators of automata, Heron of Alexandria.

Additional information

Funding

This work was supported by Gatsby Charitable Foundation (562980, 549084 for AA, 568836, 562992 for GD) and Wellcome Trust (562763 for AA, 562776 for GD)

Additional files

Supplementary Material [↗](#)

References

1. Kindler E., Krivy I. 2011) **Object-oriented simulation of systems with sophisticated control** *Int. J. Gen. Syst* **40**:313–343 <https://doi.org/10.1080/03081079.2010.539975>
2. Agha G. 1986) **Actors: A Model of Concurrent Computation in Distributed Systems** The MIT Press <https://doi.org/10.7551/mitpress/1086.001.0001>
3. Bitter R., Mohiuddin T., Nawrocki M. 2006) **LabVIEW: Advanced programming techniques** Crc Press
4. Lopes G., et al. 2015) **Bonsai: an event-based framework for processing and controlling data streams** *Front. Neuroinformatics* **9** <https://doi.org/10.3389/fninf.2015.00007>
5. Bartlett F. C., Burt C. 1933) **Remembering: A Study in Experimental and Social Psychology** *Br. J. Educ. Psychol* **3**:187–192 <https://doi.org/10.1111/j.2044-8279.1933.tb02913.x>
6. Williams L. H. 2019) **Thinking through death and employment: The automatic yet temporary use of schemata in everyday reasoning** *Eur. J. Cult. Stud* **22**:110–127 <https://doi.org/10.1177/1367549417719061>
7. Fensel D., et al., Fensel D., Şimşek U., Angele K., Huaman E., Kärle E., Panasiuk O., Toma I., Umbrich J., Wahler A. 2020) **Introduction: What Is a Knowledge Graph?** *Knowledge Graphs: Methodology, Tools and Selected Use Cases* Cham: Springer International Publishing :1–10 https://doi.org/10.1007/978-3-030-37439-6/_1
8. Semeniuta O., Falkman P. 2019) **EPypes: a framework for building event-driven data processing pipelines** *PeerJ Comput. Sci* **5**, p. e176 <https://doi.org/10.7717/peerj-cs.176>
9. Stanford Artificial Intelligence Laboratory et al 2018) **Robotic Operating System** <https://www.ros.org>
10. **vvvv - a multipurpose toolkit vvvv** <https://vvvv.org/documentation/vvvv-a-multipurposetoolkit>
11. Noone M., Mooney A. 2018) **Visual and textual programming languages: a systematic review of the literature** *J. Comput. Educ* **5**:149–174 <https://doi.org/10.1007/s40692-018-0101-5>
12. Akam T., et al. 2022) **Open-source, Python-based, hardware and software for controlling behavioural neuroscience experiments** *eLife* **11**:e67846 <https://doi.org/10.7554/eLife.67846>
13. Saunders J. L., Wehr M. 2019) **Autopilot: Automating behavioral experiments with lots of Raspberry Pis** *bioRxiv* <https://doi.org/10.1101/807693>
14. Hintjens Pieter 2013) **ZeroMQ Messaging for Many Applications, 1st** O'Reilly Media
15. Wu Y., Kirillov A., Massa F., Lo W.-Y., Girshick R. 2019) **“Detectron2.”** <https://github.com/facebookresearch/detectron2>

16. Macedo F., et al. 2022) **python-statemachine** *GitHub* <https://github.com/fgmacedo/python-statemachine>
17. Bitter R., Mohiuddin T., Nawrocki M. 2006) **LabVIEW: Advanced programming techniques** Crc Press
18. no date) **MATLAB** The MathWorks Inc
19. Thulasiraman K., Swamy M.N.S. 1992) **5.7 Acyclic Directed Graphs** *Graphs: Theory and Algorithms* John Wiley and Son :118
20. no date) <https://github.com/apache/airflow> no date <https://github.com/apache/airflow>
21. Rocklin M. 2015) **Dask: Parallel computation with blocked algorithms and task scheduling** *Proceedings of the 14th python in science conference*
22. no date) https://brodylabwiki.princeton.edu/bcontrol/index.php?title=Main_Page no date https://brodylabwiki.princeton.edu/bcontrol/index.php?title=Main_Page
23. no date) <https://github.com/fgmacedo/python-statemachine> no date <https://github.com/fgmacedo/python-statemachine>
24. no date) <https://github.com/alsivji/finite-state-machine> no date <https://github.com/alsivji/finite-state-machine>

Author information

George Dimitriadis

Sainsbury Wellcome Centre, University College London, London, United Kingdom
ORCID iD: [0000-0002-1419-1173](https://orcid.org/0000-0002-1419-1173)

For correspondence: g.dimitriadis@ucl.ac.uk

Ella Svahn

Sainsbury Wellcome Centre, University College London, London, United Kingdom,
Department of Neuroscience, Physiology and Pharmacology, University College London,
London, United Kingdom
ORCID iD: [0000-0003-1976-1531](https://orcid.org/0000-0003-1976-1531)

Andrew MacAskill

Department of Neuroscience, Physiology and Pharmacology, University College London,
London, United Kingdom
ORCID iD: [0000-0002-0196-3779](https://orcid.org/0000-0002-0196-3779)

Athena Akrami

Sainsbury Wellcome Centre, University College London, London, United Kingdom
ORCID iD: [0000-0001-5711-0903](https://orcid.org/0000-0001-5711-0903)

Editors

Reviewing Editor

Gordon Berman

Emory University, Atlanta, United States of America

Senior Editor

Kate Wassum

University of California, Los Angeles, Los Angeles, United States of America

Reviewer #2 (Public review):

Summary:

The authors provide an open-source graphic user interface (GUI) called Heron, implemented in Python, that is designed to help experimentalists to:

- (1) Design experimental pipelines and implement them in a way that is closely aligned with their mental schemata of the experiments
- (2) Execute and control the experimental pipelines with numerous interconnected hardware and software on a network.

The former is achieved by representing an experimental pipeline using a Knowledge Graph and visually representing this graph in the GUI. The latter is accomplished by using an actor model to govern the interaction among interconnected nodes through messaging, implemented using ZeroMQ. The nodes themselves execute user-supplied code in, but not limited to, Python.

Using three showcases of behavioral experiments on rats, the authors highlighted four benefits of their software design:

- (1) The knowledge graph serves as a self-documentation of the logic of the experiment, enhancing the readability and reproducibility of the experiment,
- (2) The experiment can be executed in a distributed fashion across multiple machines that each has different operating system or computing environment, such that the experiment can take advantage of hardware that sometimes can only work on a specific computer/OS, a commonly seen issue nowadays,
- (3) The users supply their own Python code for node execution that is supposed to be more friendly to those who do not have a strong programming background,
- (4) The GUI can also be used as an experiment control panel for users to control/update parameters on the fly.

Strengths:

- (1) The software is light-weight and open-source, provides a clean and easy-to-use GUI,
- (2) The software answers the need of experimentalists, particularly in the field of behavioral science, to deal with the diversity of hardware that becomes restricted to run on dedicated systems. It can also be widely adopted in many other experimental settings.
- (3) The software has a solid design that seems to be functionally reliable and useful under many conditions, demonstrated by a number of sophisticated experimental setups.
- (4) The software is well documented. The authors pay special attention to documenting the usage of the software and setting up experiments using this software.

Comments on revisions: The authors have addressed my concerns from the initial review.

<https://doi.org/10.7554/eLife.91915.2.sa1>

Author response:

The following is the authors' response to the original reviews.

Public Reviews**Reviewer #1 (Public Review):***Summary:*

The authors have created a system for designing and running experimental pipelines to control and coordinate different programs and devices during an experiment, called Heron. Heron is based around a graphical tool for creating a Knowledge Graph made up of nodes connected by edges, with each node representing a separate Python script, and each edge being a communication pathway connecting a specific output from one node to an input on another. Each node also has parameters that can be set by the user during setup and runtime, and all of this behavior is concisely specified in the code that defines each node. This tool tries to marry the ease of use, clarity, and selfdocumentation of a purely graphical system like Bonsai with the flexibility and power of a purely code-based system like Robot Operating System (ROS).

Strengths:

The underlying idea behind Heron, of combining a graphical design and execution tool with nodes that are made as straightforward Python scripts seems like a great way to get the relative strengths of each approach. The graphical design side is clear, selfexplanatory, and self-documenting, as described in the paper. The underlying code for each node tends to also be relatively simple and straightforward, with a lot of the complex communication architecture successfully abstracted away from the user. This makes it easy to develop new nodes, without needing to understand the underlying communications between them. The authors also provide useful and well-documented templates for each type of node to further facilitate this process. Overall this seems like it could be a great tool for designing and running a wide variety of experiments, without requiring too much advanced technical knowledge from the users.

The system was relatively easy to download and get running, following the directions and already has a significant amount of documentation available to explain how to use it and expand its capabilities. Heron has also been built from the ground up to easily incorporate nodes stored in separate Git repositories and to thus become a large community-driven platform, with different nodes written and shared by different groups. This gives Heron a wide scope for future utility and usefulness, as more groups use it, write new nodes, and share them with the community. With any system of this sort, the overall strength of the system is thus somewhat dependent on how widely it is used and contributed to, but the authors did a good job of making this easy and accessible for people who are interested. I could certainly see Heron growing into a versatile and popular system for designing and running many types of experiments.

Weaknesses:

(1) The number one thing that was missing from the paper was any kind of quantification of the performance of Heron in different circumstances. Several useful and illustrative examples were discussed in depth to show the strengths and flexibility of Heron, but there was no discussion or quantification of performance, timing, or latency for any of these examples. These seem like very important metrics to measure and discuss when creating a new experimental system.

Heron is practically a thin layer of obfuscation of signal passing across processes. Given its design approach it is up to the code of each Node to deal with issues of timing, synching and latency and thus up to each user to make sure the Nodes they author fulfil their experimental requirements. Having said that, Heron provides a large number of tools to allow users to optimise the generated Knowledge Graphs for their use cases. To showcase these tools, we have expanded on the third experimental example in the paper with three extra sections, two of which relate to Heron's performance and synching capabilities. One is focusing on Heron's CPU load requirements (and existing Heron tools to keep those at acceptable limits) and another focusing on post experiment synchronisation of all the different data sets a multi Node experiment generates.

(2) After downloading and running Heron with some basic test Nodes, I noticed that many of the nodes were each using a full CPU core on their own. Given that this basic test experiment was just waiting for a keypress, triggering a random number generator, and displaying the result, I was quite surprised to see over 50% of my 8-core CPU fully utilized. I don't think that Heron needs to be perfectly efficient to accomplish its intended purpose, but I do think that some level of efficiency is required. Some optimization of the codebase should be done so that basic tests like this can run with minimal CPU utilization. This would then inspire confidence that Heron could deal with a real experiment that was significantly more complex without running out of CPU power and thus slowing down.

The original Heron allowed the OS to choose how to manage resources over the required process. We were aware that this could lead to significant use of CPU time, as well as occasionally significant drop of packets (which was dependent on the OS and its configuration). This drop happened mainly when the Node was running a secondary process (like in the Unity game process in the 3rd example). To mitigate these problems, we have now implemented a feature allowing the user to choose the CPU that each Node's worker function runs on as well as any extra processes the worker process initialises. This is accessible from the Saving secondary window of the node. This stops the OS from swapping processes between CPUs and eliminates the dropping of packages due to the OS behaviour. It also significantly reduces the utilised CPU time. To showcase this, we initially run the simple example mentioned by the reviewer. The computer running only background services was using 8% of CPU (8 cores). With Heron GUI running but with no active Graph, the CPU usage went to 15%. With the Graph running and Heron's processes running on OS attributed CPU cores, the total CPU was at 65% (so very close to the reviewer's 50%). By choosing a different CPU core for each of the three worker processes the CPU went down to 47% and finally when all processes were forced to run on the same CPU core the CPU load dropped to 30%. So, Heron in its current implementation running its GUI and 3 Nodes takes 22% of CPU load. This is still not ideal but is a consequence of the overhead of running multiple processes vs multiple threads. We believe that, given Heron's latest optimisation, offering more control of system management to the user, the benefits of multi process applications outweigh this hit in system resources.

We have also increased the scope of the third example we provide in the paper and there we describe in detail how a full-scale experiment with 15 Nodes (which is the upper limit of number of Nodes usually required in most experiments) impacts CPU load.

Finally, we have added on Heron's roadmap projects extra tasks focusing only on optimisation (profiling and using Numba for the time critical parts of the Heron code).

(3) I was also surprised to see that, despite being meant specifically to run on and connect diverse types of computer operating systems and being written purely in Python, the Heron Editor and GUI must be run on Windows. This seems like an unfortunate and

unnecessary restriction, and it would be great to see the codebase adjusted to make it fully crossplatform-compatible.

This point was also mentioned by reviewer 2. This was a mistake on our part and has now been corrected in the paper. Heron (GUI and underlying communication functionality) can run on any machine that the underlying python libraries run, which is Windows, Linux (both for x86 and Arm architectures) and MacOS. We have tested it on Windows (10 and 11, both x64), Linux PC (Ubuntu 20.04.6, x64) and Raspberry Pi 4 (Debian GNU/Linux 12 (bookworm), aarch64). The Windows and Linux versions of Heron have undergone extensive debugging and all of the available Nodes (that are not OS specific) run on those two systems. We are in the process of debugging the Nodes' functionality for RasPi. The MacOS version, although functional requires further work to make sure all of the basic Nodes are functional (which is not the case at the moment). We have also updated our manuscript (Multiple machines, operating systems and environments) to include the above information.

(4) Lastly, when I was running test experiments, sometimes one of the nodes, or part of the Heron editor itself would throw an exception or otherwise crash. Sometimes this left the Heron editor in a zombie state where some aspects of the GUI were responsive and others were not. It would be good to see a more graceful full shutdown of the program when part of it crashes or throws an exception, especially as this is likely to be common as people learn to use it. More problematically, in some of these cases, after closing or force quitting Heron, the TCP ports were not properly relinquished, and thus restarting Heron would run into an "address in use" error. Finding and killing the processes that were still using the ports is not something that is obvious, especially to a beginner, and it would be great to see Heron deal with this better. Ideally, code would be introduced to carefully avoid leaving ports occupied during a hard shutdown, and furthermore, when the address in use error comes up, it would be great to give the user some idea of what to do about it.

A lot of effort has been put into Heron to achieve graceful shut down of processes, especially when these run on different machines that do not know when the GUI process has closed. The code that is being suggested to avoid leaving ports open has been implemented and this works properly when processes do not crash (Heron is terminated by the user) and almost always when there is a bug in a process that forces it to crash. In the version of Heron available during the reviewing process there were bugs that caused the above behaviour (Node code hanging and leaving zombie processes) on MacOS systems. These have now been fixed. There are very seldom instances though, especially during Node development, that crashing processes will hang and need to be terminated manually. We have taken on board the reviewer's comments that users should be made more aware of these issues and have also described this situation in the Debugging part of Heron's documentation. There we explain the logging and other tools Heron provides to help users debug their own Nodes and how to deal with hanging processes.

Heron is still in alpha (usable but with bugs) and the best way to debug it and iron out all the bugs in all use cases is through usage from multiple users and error reporting (we would be grateful if the errors the reviewer mentions could be reported in Heron's github Issues page). We are always addressing and closing any reported errors, since this is the only way for Heron to transition from alpha to beta and eventually to production code quality.

Overall I think that, with these improvements, this could be the beginning of a powerful and versatile new system that would enable flexible experiment design with a relatively low technical barrier to entry. I could see this system being useful to many different labs and fields.

We thank the reviewer for positive and supportive words and for the constructive feedbacks. We believe we have now addressed all the raised concerns.

Reviewer #2 (Public Review):

Summary:

The authors provide an open-source graphic user interface (GUI) called Heron, implemented in Python, that is designed to help experimentalists to

(1) design experimental pipelines and implement them in a way that is closely aligned with their mental schemata of the experiments,

(2) execute and control the experimental pipelines with numerous interconnected hardware and software on a network.

The former is achieved by representing an experimental pipeline using a Knowledge Graph and visually representing this graph in the GUI. The latter is accomplished by using an actor model to govern the interaction among interconnected nodes through messaging, implemented using ZeroMQ. The nodes themselves execute user-supplied code in, but not limited to, Python.

Using three showcases of behavioral experiments on rats, the authors highlighted three benefits of their software design:

(1) the knowledge graph serves as a self-documentation of the logic of the experiment, enhancing the readability and reproducibility of the experiment,

(2) the experiment can be executed in a distributed fashion across multiple machines that each has a different operating system or computing environment, such that the experiment can take advantage of hardware that sometimes can only work on a specific computer/OS, a commonly seen issue nowadays,

(3) he users supply their own Python code for node execution that is supposed to be more friendly to those who do not have a strong programming background.

Strengths:

(1) The software is light-weight and open-source, provides a clean and easy-to-use GUI,

(2) The software answers the need of experimentalists, particularly in the field of behavioral science, to deal with the diversity of hardware that becomes restricted to run on dedicated systems.

(3) The software has a solid design that seems to be functionally reliable and useful under many conditions, demonstrated by a number of sophisticated experimental setups.

(4) The software is well documented. The authors pay special attention to documenting the usage of the software and setting up experiments using this software.

Weaknesses:

(1) While the software implementation is solid and has proven effective in designing the experiment showcased in the paper, the novelty of the design is not made clear in the manuscript. Conceptually, both the use of graphs and visual experimental flow design have been key features in many widely used softwares as suggested in the background section of the manuscript. In particular, contrary to the authors' claim that only pre-

defined elements can be used in Simulink or LabView, Simulink introduced MATLAB Function Block back in 2011, and Python code can be used in LabView since 2018. Such customization of nodes is akin to what the authors presented.

In the Heron manuscript we have provided an extensive literature review of existing systems from which Heron has borrowed ideas. We never wished to say that graphs and visual code is what sets Heron apart since these are technologies predating Heron by many years and implemented by a large number of software. We do not believe also that we have mentioned that LabView or Simulink can utilise only predefined nodes. What we have said is that in such systems (like LabView, Simulink and Bonsai) the focus of the architecture is on prespecified low level elements while the ability for users to author their own is there but only as an afterthought. The difference with Heron is that in the latter the focus is on the users developing their own elements. One could think of LabView style software as node-based languages (with low level visual elements like loops and variables) that also allow extra scripting while Heron is a graphical wrapper around python where nodes are graphical representations of whole processes. To our knowledge there is no other software that allows the very fast generation of graphical elements representing whole processes whose communication can also be defined graphically. Apart from this distinction, Heron also allows a graphical approach to writing code for processes that span different machines which again to our knowledge is a novelty of our approach and one of its strongest points towards ease of experimental pipeline creation (without sacrificing expressivity).

(2) The authors claim that the knowledge graph can be considered as a self-documentation of an experiment. I found it to be true to some extent. Conceptually it's a welcoming feature and the fact that the same visualization of the knowledge graph can be used to run and control experiments is highly desirable (but see point 1 about novelty). However, I found it largely inadequate for a person to understand an experiment from the knowledge graph as visualized in the GUI alone. While the information flow is clear, and it seems easier to navigate a codebase for an experiment using this method, the design of the GUI does not make it a one-stop place to understand the experiment. Take the Knowledge Graph in Supplementary Figure 2B as an example, it is associated with the first showcase in the result section highlighting this self-documentation capability. I can see what the basic flow is through the disjoint graph where 1) one needs to press a key to start a trial, and 2) camera frames are saved into an avi file presumably using FFMPEG. Unfortunately, it is not clear what the parameters are and what each block is trying to accomplish without the explanation from the authors in the main text. Neither is it clear about what the experiment protocol is without the help of Supplementary Figure 2A.

In my opinion, text/figures are still key to documenting an experiment, including its goals and protocols, but the authors could take advantage of the fact that they are designing a GUI where this information, with properly designed API, could be easily displayed, perhaps through user interaction. For example, in Local Network -> Edit IPs/ports in the GUI configuration, there is a good tooltip displaying additional information for the "password" entry. The GUI for the knowledge graph nodes can very well utilize these tooltips to show additional information about the meaning of the parameters, what a node does, etc, if the API also enforces users to provide this information in the form of, e.g., Python docstrings in their node template. Similarly, this can be applied to edges to make it clear what messages/data are communicated between the nodes. This could greatly enhance the representation of the experiment from the Knowledge graph.

In the first showcase example in the paper “Probabilistic reversal learning.

Implementation as self-documentation” we go through the steps that one would follow in order to understand the functionality of an experiment through Heron’s Knowledge Graph.

The Graph is not just the visual representation of the Nodes in the GUI but also their corresponding code bases. We mention that the way Heron's API limits the way a Node's code is constructed (through an Actor based paradigm) allows for experimenters to easily go to the code base of a specific Node and understand its 2 functions (initialisation and worker) without getting bogged down in the code base of the whole Graph (since these two functions never call code from any other Nodes). Newer versions of Heron facilitate this easy access to the appropriate code by also allowing users to attach to Heron their favourite IDE and open in it any Node's two scripts (worker and com) when they double click on the Node in Heron's GUI. On top of this, Heron now (in the versions developed as answers to the reviewers' comments) allows Node creators to add extensive comments on a Node but also separate comments on the Node's parameters and input and output ports. Those can be seen as tooltips when one hovers over the Node (a feature that can be turned off or on by the Info button on every Node).

As Heron stands at the moment we have not made the claim that the Heron GUI is the full picture in the self-documentation of a Graph. We take note though the reviewer's desire to have the GUI be the only tool a user would need to use to understand an experimental implementation. The solution to this is the same as the one described by the reviewer of using the GUI to show the user the parts of the code relevant to a specific Node without the user having to go to a separate IDE or code editor. The reason this has not been implemented yet is the lack of a text editor widget in the underlying gui library (DearPyGUI). This is in their roadmap for their next large release and when this exists we will use it to implement exactly the idea the reviewer is suggesting, but also with the capability to not only read comments and code but also directly edit a Node's code (see Heron's roadmap). Heron's API at the moment is ideal for providing such a text editor straight from the GUI.

(3) The design of Heron was primarily with behavioral experiments in mind, in which highly accurate timing is not a strong requirement. Experiments in some other areas that this software is also hoping to expand to, for example, electrophysiology, may need very strong synchronization between apparatus, for example, the record timing and stimulus delivery should be synced. The communication mechanism implemented in Heron is asynchronous, as I understand it, and the code for each node is executed once upon receiving an event at one or more of its inputs. The paper, however, does not include a discussion, or example, about how Heron could be used to address issues that could arise in this type of communication. There is also a lack of information about, for example, how nodes handle inputs when their ability to execute their work function cannot keep up with the frequency of input events. Does the publication/subscription handle the queue intrinsically? Will it create problems in real-time experiments that make multiple nodes run out of sync? The reader could benefit from a discussion about this if they already exist, and if not, the software could benefit from implementing additional mechanisms such that it can meet the requirements from more types of experiments.

In order to address the above lack of explanation (that also the first reviewer pointed out) we expanded the third experimental example in the paper with three more sections. One focuses solely on explaining how in this example (which acquires and saves large amounts of data from separate Nodes running on different machines) one would be able to time align the different data packets generated in different Nodes to each other. The techniques described there are directly implementable on experiments where the requirements of synching are more stringent than the behavioural experiment we showcase (like in ephys experiments).

Regarding what happens to packages when the worker function of a Node is too slow to handle its traffic, this is mentioned in the paper (Code architecture paragraph): "Heron is designed to have no message buffering, thus automatically dropping any messages that come into a Node's inputs while the Node's worker function is still running." This is also explained in more detail in Heron's documentation. The reasoning for a no buffer system (as described

in the documentation) is that for the use cases Heron is designed to handle we believe there is no situation where a Node would receive large amounts of data in bursts while very little data during the rest of the time (in which case a buffer would make sense). Nodes in most experiments will either be data intensive but with a constant or near constant data receiving speed (e.g. input from a camera or ephys system) or will have variable data load reception but always with small data loads (e.g. buttons). The second case is not an issue and the first case cannot be dealt with a buffer but with the appropriate code design, since buffering data coming in a Node too slow for its input will just postpone the inevitable crash. Heron's architecture principle in this case is to allow these 'mistakes' (i.e. package dropping) to happen so that the pipeline continues to run and transfer the responsibility of making Nodes fast enough to the author of each Node. At the same time Heron provides tools (see the Debugging section of the documentation and the time alignment paragraph of the "Rats playing computer games" example in the manuscript) that make it easy to detect package drops and either correct them or allow them but also allow time alignment between incoming and outgoing packets. In the very rare case where a buffer is required Heron's do-it-yourself logic makes it easy for a Node developer to implement their own Node specific buffer.

(4) The authors mentioned in "Heron GUI's multiple uses" that the GUI can be used as an experimental control panel where the user can update the parameters of the different Nodes on the fly. This is a very useful feature, but it was not demonstrated in the three showcases. A demonstration could greatly help to support this claim.

As the reviewer mentions, we have found Heron's GUI double role also as an experimental on-line controller a very useful capability during our experiments. We have expanded the last experimental example to also showcase this by showing how on the "Rats playing computer games" experiment we used the parameters of two Nodes to change the arena's behaviour while the experiment was running, depending on how the subject was behaving at the time (thus exploring a much larger set of parameter combinations, faster during exploratory periods of our shaping protocols construction).

(5) The API for node scripts can benefit from having a better structure as well as having additional utilities to help users navigate the requirements, and provide more guidance to users in creating new nodes. A more standard practice in the field is to create three abstract Python classes, Source, Sink, and Transform that dictate the requirements for initialisation, work_function, and on_end_of_life, and provide additional utility methods to help users connect between their code and the communication mechanism. They can be properly docstringed, along with templates. In this way, the com and worker scripts can be merged into a single unified API. A simple example that can cause confusion in the worker script is the "worker_object", which is passed into the initialise function. It is unclear what this object this variable should be, and what attributes are available without looking into the source code. As the software is also targeting those who are less experienced in programming, setting up more guidance in the API can be really helpful. In addition, the self-documentation aspect of the GUI can also benefit from a better structured API as discussed in point 2 above.

The reviewer is right that using abstract classes to expose to users the required API would be a more standard practice. The reason we did not choose to do this was to keep Heron easily accessible to entry level Python programmers who do not have familiarity yet with object oriented programming ideas. So instead of providing abstract classes we expose only the implementation of three functions which are part of the worker classes but the classes themselves are not seen by the users of the API. The point about the users' accessibility to more information regarding a few objects used in the API (the worker object for example) has been taken on board and we have now addressed this by type hinting all these objects both in

the templates and more importantly in the automatically generated code that Heron now creates when a user chooses to create a Node graphically (a feature of Heron not present in the version available in the initial submission of this manuscript).

(6) The authors should provide more pre-defined elements. Even though the ability for users to run arbitrary code is the main feature, the initial adoption of a codebase by a community, in which many members are not so experienced with programming, is the ability for them to use off-the-shelf components as much as possible. I believe the software could benefit from a suite of commonly used Nodes.

There are currently 12 Node repositories in the Heron-repositories project on Github with more than 30 Nodes, 20 of which are general use (not implementing a specific experiment logic). This list will continue to grow but we fully appreciate the truth of the reviewer's comment that adoption will depend on the existence of a large number of commonly used Nodes (for example Numpy, and OpenCV Nodes) and are working towards this goal.

(7) It is not clear to me if there is any capability or utilities for testing individual nodes without invoking a full system execution. This would be critical when designing new experiments and testing out each component.

There is no capability to run the code of an individual Node outside Heron's GUI. A user could potentially design and test parts of the Node before they get added into a Node but we have found this to be a highly inefficient way of developing new Nodes. In our hands the best approach for Node development was to quickly generate test inputs and/or outputs using the "User Defined Function 1I 1O" Node where one can quickly write a function and make it accessible from a Node. Those test outputs can then be pushed in the Node under development or its outputs can be pushed in the test function, to allow for incremental development without having to connect it to the Nodes it would be connected in an actual pipeline. For example, one can easily create a small function that if a user presses a key will generate the same output (if run from a "User Defined Function 1I 1O" Node) as an Arduino Node reading some buttons. This output can then be passed into an experiment logic Node under development that needs to do something with this input. In this way during a Node development Heron allows the generation of simulated hardware inputs and outputs without actually running the actual hardware. We have added this way of developing Nodes also in our manuscript (Creating a new Node).

Reviewer #3 (Public Review):

Summary:

The authors present a Python tool, Heron, that provides a framework for defining and running experiments in a lab setting (e.g. in behavioural neuroscience). It consists of a graphical editor for defining the pipeline (interconnected nodes with parameters that can pass data between them), an API for defining the nodes of these pipelines, and a framework based on ZeroMQ, responsible for the overall control and data exchange between nodes. Since nodes run independently and only communicate via network messages, an experiment can make use of nodes running on several machines and in separate environments, including on different operating systems.

Strengths:

As the authors correctly identify, lab experiments often require a hodgepodge of separate hardware and software tools working together. A single, unified interface for defining these connections and running/supervising the experiment, together with flexibility in defining the individual subtasks (nodes) is therefore a very welcome

approach. The GUI editor seems fairly intuitive, and Python as an accessible programming environment is a very sensible choice. By basing the communication on the widely used ZeroMQ framework, they have a solid base for the required non-trivial coordination and communication. Potential users reading the paper will have a good idea of how to use the software and whether it would be helpful for their own work. The presented experiments convincingly demonstrate the usefulness of the tool for realistic scientific applications.

Weaknesses:

(1) In my opinion, the authors somewhat oversell the reproducibility and "selfdocumentation" aspect of their solution. While it is certainly true that the graph representation gives a useful high-level overview of an experiment, it can also suffer from the same shortcomings as a "pure code" description of a model - if a user gives their nodes and parameters generic/unhelpful names, reading the graph will not help much.

This is a problem that to our understanding no software solution can possibly address. Yet having a visual representation of how different inputs and outputs connect to each other we argue would be a substantial benefit in contrast to the case of "pure code" especially when the developer of the experiment has used badly formatted variable names.

(2) Making the link between the nodes and the actual code is also not straightforward, since the code for the nodes is spread out over several directories (or potentially even machines), and not directly accessible from within the GUI.

This is not accurate. The obligatory code of a Node always exists within a single folder and Heron's API makes it rather cumbersome to spread scripts relating to a Node across separate folders. The Node folder structure can potentially be copied over different machines but this is why Heron is tightly integrated with git practices (and even politely asks the user with popup windows to create git repositories of any Nodes they create whilst using Heron's automatic Node generator system). Heron's documentation is also very clear on the folder structure of a Node which keeps the required code always in the same place across machines and more importantly across experiments and labs. Regarding the direct accessibility of the code from the GUI, we took on board the reviewers' comments and have taken the first step towards correcting this. Now one can attach to Heron their favourite IDE and then they can double click on any Node to open its two main scripts (com and worker) in that IDE embedded in whatever code project they choose (also set in Heron's settings windows). On top of this, Heron now allows the addition of notes both for a Node and for all its parameters, inputs and outputs which can be viewed by hovering the mouse over them on the Nodes' GUIs. The final step towards GUI-code integration will be to have a Heron GUI code editor but this is something that has to wait for further development from Heron's underlying GUI library DearPyGUI.

(3) The authors state that "[Heron's approach] confers obvious benefits to the exchange and reproducibility of experiments", but the paper does not discuss how one would actually exchange an experiment and its parameters, given that the graph (and its json representation) contains user-specific absolute filenames, machine IP addresses, etc, and the parameter values that were used are stored in general data frames, potentially separate from the results. Neither does it address how a user could keep track of which versions of files were used (including Heron itself).

Heron's Graphs, like any experimental implementation, must contain machine specific strings. These are accessible either from Heron's GUI when a Graph json file is opened or from the json file itself. Heron in this regard does not do anything different to any other

software, other than saving the graphs into human readable json files that users can easily manipulate directly.

Heron provides a method for users to save every change of the Node parameters that might happen during an experiment so that it can be fully reproduced. The dataframes generated are done so in the folders specified by the user in each of the Nodes (and all those paths are saved in the json file of the Graph). We understand that Heron offers a certain degree of freedom to the user (Heron's main reason to exist is exactly this versatility) to generate data files wherever they want but makes sure every file path gets recorded for subsequent reproduction. So, Heron behaves pretty much exactly like any other open source software. What we wanted to focus on as the benefits of Heron on exchange and reproducibility was the ability of experimenters to take a Graph from another lab (with its machine specific file paths and IP addresses) and by examining the graphical interface of it to be able to quickly tweak it to make it run on their own systems. That is achievable through the fact that a Heron experiment will be constructed by a small amount of Nodes (5 to 15 usually) whose file paths can be trivially changed in the GUI or directly in the json file while the LAN setup of the machines used can be easily reconstructed from the information saved in the secondary GUIs.

Where Heron needs to improve (and this is a major point in Heron's roadmap) is the need to better integrate the different saved experiments with the git versions of Heron and the Nodes that were used for that specific save. This, we appreciate is very important for full reproducibility of the experiment and it is a feature we will soon implement. More specifically users will save together with a graph the versions of all the used repositories and during load the code base utilised will come from the recorded versions and not from the current head of the different repositories. This is a feature that we are currently working on now and as our roadmap suggests will be implemented by the release of Heron 1.0.

(4) Another limitation that in my opinion is not sufficiently addressed is the communication between the nodes, and the effect of passing all communications via the host machine and SSH. What does this mean for the resulting throughput and latency - in particular in comparison to software such as Bonsai or Autopilot? The paper also states that "Heron is designed to have no message buffering, thus automatically dropping any messages that come into a Node's inputs while the Node's worker function is still running." - it seems to be up to the user to debug and handle this manually?

There are a few points raised here that require addressing. The first is Heron's requirement to pass all communication through the main (GUI) machine. We understand (and also state in the manuscript) that this is a limitation that needs to be addressed. We plan to do this by adding to Heron the feature of running headless (see our roadmap). This will allow us to run whole Heron pipelines in a second machine which will communicate with the main pipeline (run on the GUI machine) with special Nodes. That will allow experimenters to define whole pipelines on secondary machines where the data between their Nodes stay on the machine running the pipeline. This is an important feature for Heron and it will be one of the first features to be implemented next (after the integration of the saving system with git).

The second point is regarding Heron's throughput latency. In our original manuscript we did not have any description of Heron's capabilities in this respect and both other reviewers mentioned this as a limitation. As mentioned above, we have now addressed this by adding a section to our third experimental example that fully describes how much CPU is required to run a full experimental pipeline running on two machines and utilising also non python code executables (a Unity game). This gives an overview of how heavy pipelines can run on normal computers given adequate optimisation and utilising Heron's feature of forcing some Nodes to run their Worker processes on a specific core. At the same time, Heron's use of 0MQ protocol makes sure there are no other delays or speed limitations to message passing. So,

message passing within the same machine is just an exchange of memory pointers while messages passing between different machines face the standard speed limitations of the Local Access Network's ethernet card speeds.

Finally, regarding the message dropping feature of Heron, as mentioned above this is an architectural decision given the use cases of message passing we expect Heron to come in contact with. For a full explanation of the logic here please see our answer to the 3rd comment by Reviewer 2.

(5) As a final comment, I have to admit that I was a bit confused by the use of the term "Knowledge Graph" in the title and elsewhere. In my opinion, the Heron software describes "pipelines" or "data workflows", not knowledge graphs - I'd understand a knowledge graph to be about entities and their relationships. As the authors state, it is usually meant to make it possible to "test propositions against the knowledge and also create novel propositions" - how would this apply here?

We have described Heron as a Knowledge Graph instead of a pipeline, data workflow or computation graph in order to emphasise Heron's distinct operation in contrast to what one would consider a standard pipeline and data workflow generated by other visual based software (like LabView and Bonsai). This difference exists on what a user should think of as the base element of a graph, i.e. the Node. In all other visual programming paradigms, the Node is defined as a low-level computation, usually a language keyword, language flow control or some simple function. The logic in this case is generated by composing together the visual elements (Nodes). In Heron the Node is to be thought of as a process which can be of arbitrary complexity and the logic of the graph is composed by the user both within each Node and by the way the Nodes are combined together. This is an important distinction in Heron's basic operation logic and it is we argue the main way Heron allows flexibility in what can be achieved while retaining ease of graph composition (by users defining their own level of complexity and functionality encompassed within each Node). We have found that calling this approach a computation graph (which it is) or a pipeline or data workflow would not accentuate this difference. The term Knowledge Graph was the most appropriate as it captures the essence of variable information complexity (even in terms of length of shortest string required) defined by a Node.

Recommendations for the authors:

Reviewer #1 (Recommendations For The Authors):

- No buffering implies dropped messages when a node is busy. It seems like this could be very problematic for some use cases...

This is a design principle of Heron. We have now provided a detailed explanation of the reasoning behind it in our answer to Reviewer 2 (Paragraph 3) as well as in the manuscript.

- How are ssh passwords stored, and is it secure in some way or just in plain text?

For now they are plain text in an unencrypted file that is not part of the repo (if one gets Heron from the repo). Eventually, we would like to go to private/public key pairs but this is not a priority due to the local nature of Heron's use cases (all machines in an experiment are expected to connect in a LAN).

Minor notes / copyedits:

- Figure 2A: right and left seem to be reversed in the caption.

They were. This is now fixed.

- Figure 2B: the text says that proof of life messages are sent to each worker process but in the figure, it looks like they are published by the workers? Also true in the online documentation.

The Figure caption was wrong. This is now fixed.

- psutil package is not included in the requirements for GitHub

We have now included psutil in the requirements.

- GitHub readme says Python >=3.7 but Heron will not run as written without python >= 3.9 (which is alluded to in the paper)

The new Heron updates require Python 3.11. We have now updated GitHub and the documentation to reflect this.

- The paper mentions that the Heron editor must be run on Windows, but this is not mentioned in the Github readme.

This was an error in the manuscript that we have now corrected.

- It's unclear from the readme/manual how to remove a node from the editor once it's been added.

We have now added an X button on each Node to complement the Del button on the keyboard (for MacOS users that do not have this button most of the times).

- The first example experiment is called the Probabilistic Reversal Learning experiment in text, but the uncertainty experiment in the supplemental and on GitHub.

We have now used the correct name (Probabilistic Reversal Learning) in both the supplemental material and on GitHub

- Since Python >=3.9 is required, consider using fstrings instead of str.format for clarity in the codebase

Thank you for the suggestion. Latest Heron development has been using f strings and we will do a refactoring in the near future.

- Grasshopper cameras can run on linux as well through the spinnaker SDK, not just Windows.

Fixed in the manuscript.

- Figure 4: Square and star indicators are unclear.

Increased the size of the indicators to make them clear.

- End of page 9: "an of the self" presumably a typo for "off the shelf"?

Corrected.

- Page 10 first paragraph. "second root" should be "second route"

Corrected.

- When running Heron, the terminal constantly spams Blowfish encryption deprecation warnings, making it difficult to see the useful messages.

The solution to this problem is to either update paramiko or install Heron through pip. This possible issue is mentioned in the documentation.

- Node input /output hitboxes in the GUI are pretty small. If they could be bigger it would make it easier to connect nodes reliably without mis-clicks.

We have redone the Node GUI, also increasing the size of the In/Out points.

Reviewer #2 (Recommendations For The Authors):

(1) There are quite a few typos in the manuscript, for example: "one can accessess the code", "an of the self", etc.

Thanks for the comment. We have now screened the manuscript for possible typos.

(2) Heron's GUI can only run on Windows! This seems to be the opposite of the key argument about the portability of the experimental setup.

As explained in the answers to Reviewer 1, Heron can run on most machines that the underlying python libraries run, i.e. Windows and Linux (both for x86 and Arm architectures). We have tested it on Windows (10 and 11, both x64), Linux PC (Ubuntu 20.04.6, x64) and Raspberry Pi 4 (Debian GNU/Linux 12 (bookworm), aarch64). We have now revised the manuscript and the GitHub repo to reflect this.

(3) Currently, the output is displayed along the left edge of the node, but the yellow dot connector is on the right. It would make more sense to have the text displayed next to the connectors.

We have redesigned the Node GUI and have now placed the Out connectors on the right side of the Node.

(4) The edges are often occluded by the nodes in the GUI. Sometimes it leads to some confusion, particularly when the number of nodes is large, e.g., Fig 4.

This is something that is dependent on the capabilities of the DearPyGUI module. At the moment there is no way to control the way the edges are drawn.

Reviewer #3 (Recommendations For The Authors):

A few comments on the software and the documentation itself:

- From a software engineering point of view, the implementation seems to be rather immature. While I get the general appeal of "no installation necessary", I do not think that installing dependencies by hand and cloning a GitHub repository is easier than installing a standard package.

We have now added a pip install capability which also creates a Heron command line command to start Heron with.

-The generous use of global variables to store state (minor point, given that all nodes run in different processes), boilerplate code that each node needs to repeat, and the absence of any kind of automatic testing do not give the impression of a very mature software (case in point: I had to delete a line from editor.py to be able to start it on a non-Windows system).

As mentioned, the use of global variables in the worker scripts is fine partly due to the multi process nature of the development and we have found it is a friendly approach to Matlab users who are just starting with Python (a serious consideration for Heron). Also, the parts of the code that would require a singleton (the Editor for example) are treated as scripts with global variables while the parts that require the construction of objects are fully embedded in classes (the Node for example). A future refactoring might make also all the parts of the code not seen by the user fully object oriented but this is a decision with pros and cons needing to be weighted first.

Absence of testing is an important issue we recognise but Heron is a GUI app and nontrivial unit tests would require some keystroke/mouse movement emulator (like QTest of pytest-qt for QT based GUIs). This will be dealt with in the near future (using more general solutions like PyAutoGUI) but it is something that needs a serious amount of effort (quite a bit more than writing unit tests for non GUI based software) and more importantly it is nowhere as robust as standard unit tests (due to the variable nature of the GUI through development) making automatic test authoring an almost as laborious a process as the one it is supposed to automate.

- From looking at the examples, I did not quite see why it is necessary to write the ..._com.py scripts as Python files, since they only seem to consist of boilerplate code and variable definitions. Wouldn't it be more convenient to represent this information in configuration files (e.g. yaml or toml)?

The com is not a configuration file, it is a script that launches the communication process of the Node. We could remove the variable definitions to a separate toml file (which then the com script would have to read). The pros and cons of such a set up should be considered in a future refactoring.

Minor comments for the paper:

- p.7 (top left): "through its return statement" - the worker loop is an infinite loop that forwards data with a return statement?

This is now corrected. The worker loop is an infinite loop and does not return anything but at each iteration pushes data to the Nodes output.

- p.9 (bottom right): "of the self" → "off-the-shelf"

Corrected.

- p.10 (bottom left): "second root" → "second route"

Corrected.

- *Supplementary Figure 3: Green star and square seem to be swapped (the green star on top is a camera image and the green star on the bottom is value visualization - inversely for the green square).*

The star and square have been swapped around.

- *Caption Supplementary Figure 4 (end): "rashes to receive" → "rushes to receive"*

Corrected.

<https://doi.org/10.7554/eLife.91915.2.sa0>